

Machine Learning Workshop | L04

ANN in depth | Convolutional Neural Network

Recap

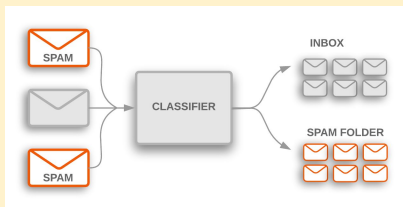
Artificial Intelligence

Any techniques that enables computers to mimic human behaviour



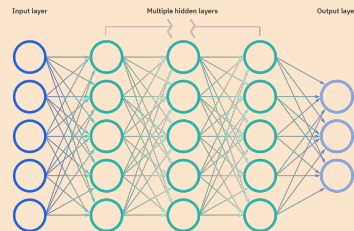
Machine Learning

Ability to learn without explicitly being programmed

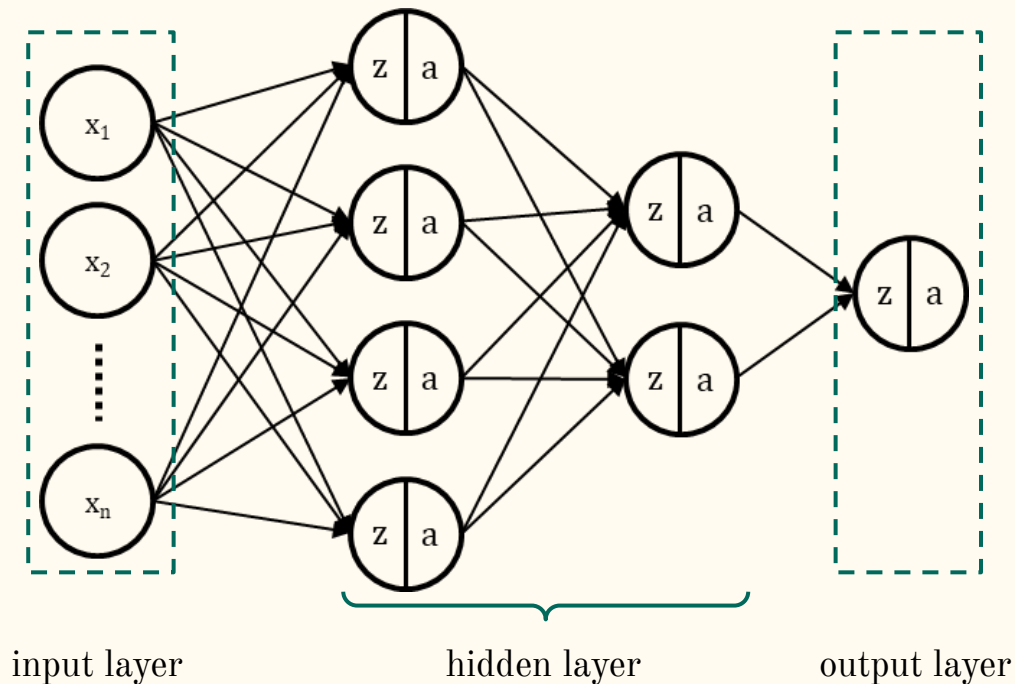
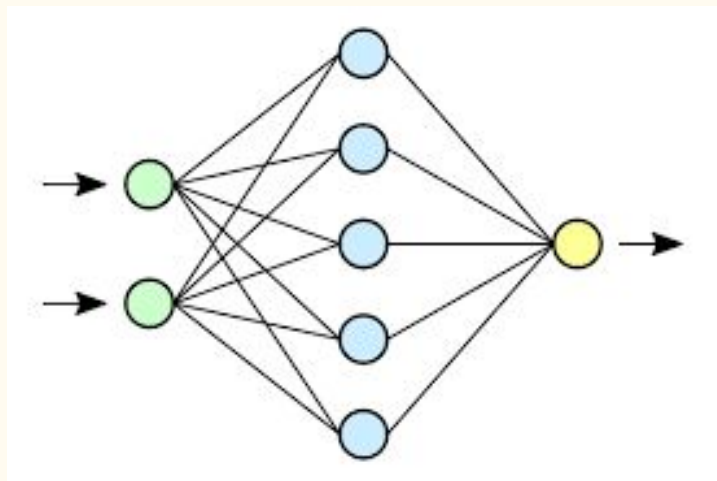


Deep Learning

Extract patterns from data using neural networks



Recap: Structure of Neural Network



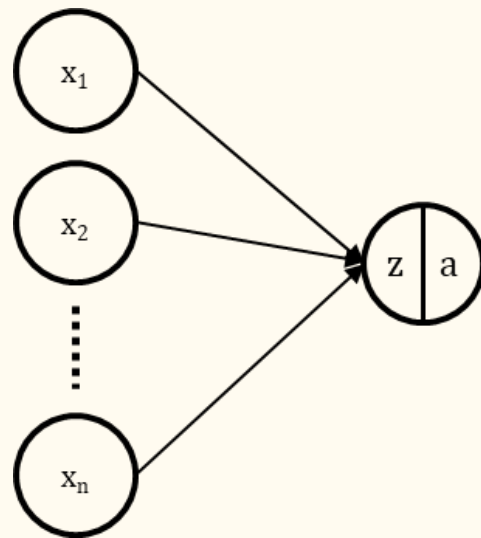
From regression to neural network

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = y$$

Multiple regression

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = z$$
$$g(z) = a$$

Neural network



Recap : Activation Function

Activation function is just a function to obtain the output of a node / neuron
Aka Transfer function

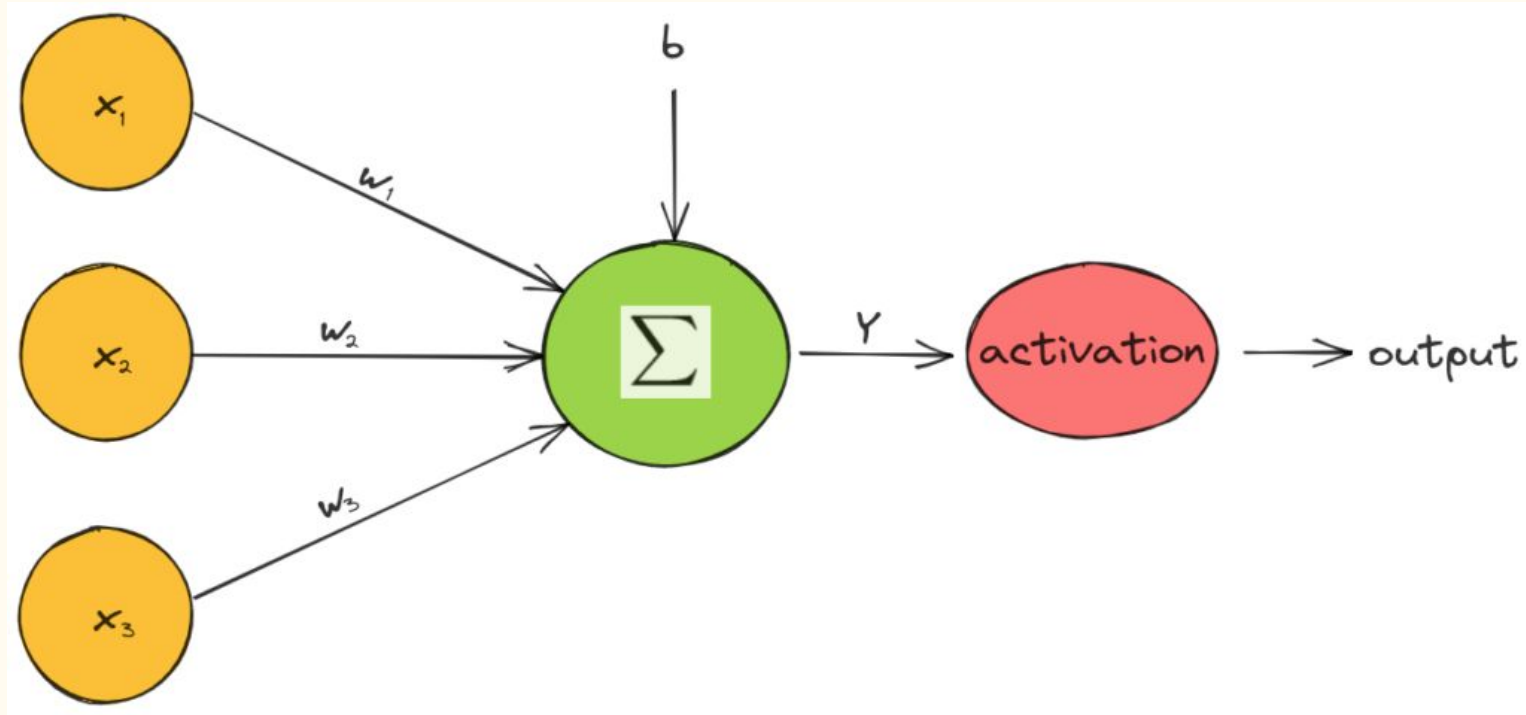
Input : all neuron from previous layer $[wa + b]$

process : activation function / transfer z to a

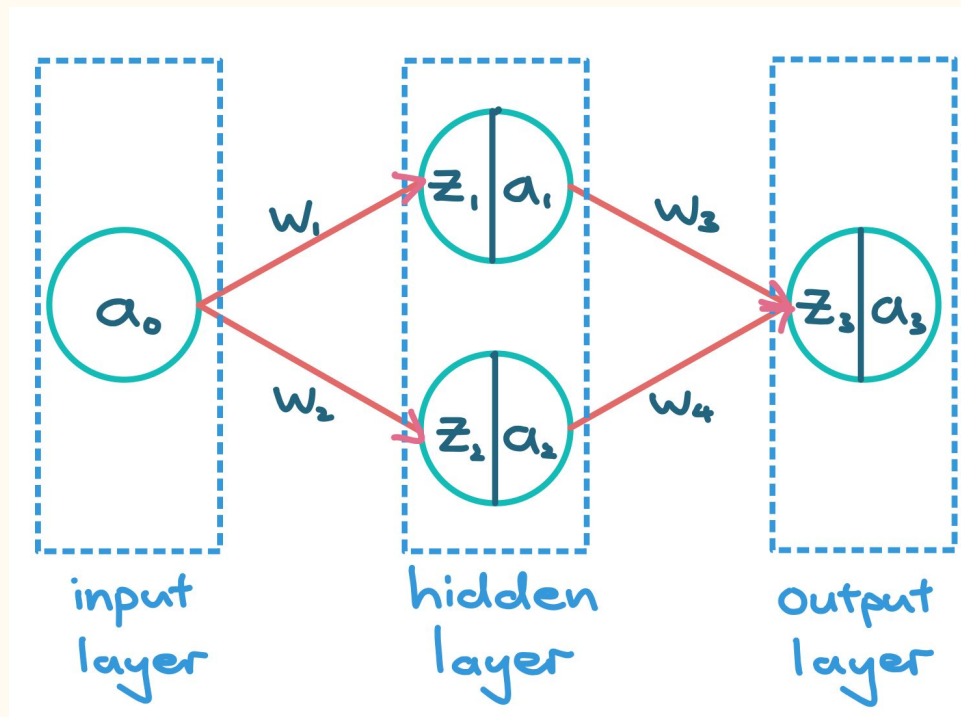
Output : Hidden layer - simply the output of neuron

Output layer - the 'ultimate result' of the model

Activation Function



Forward Pass



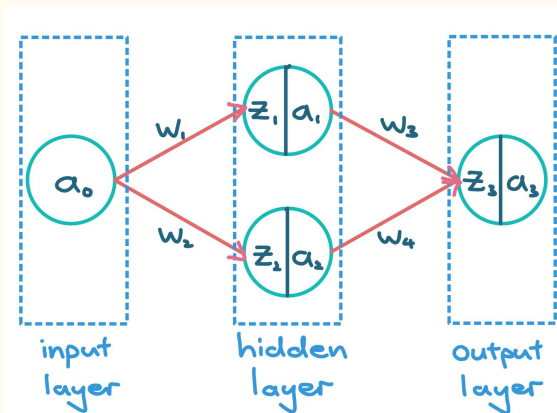
a_0 : input feature

$z|a$: 過 Activation

$a = g(z)$
Activation Function.

$\hat{y}$: output / prediction

Forward Pass

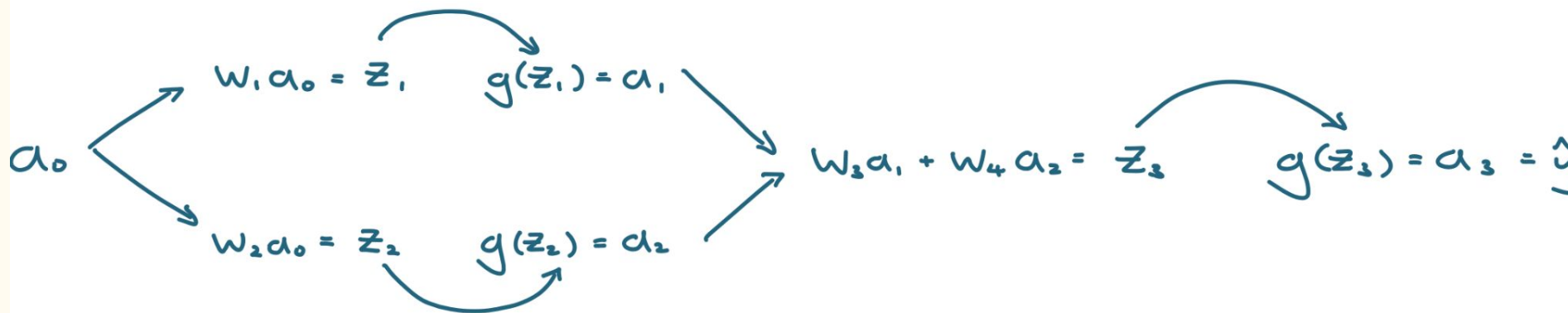


a_0 : input feature

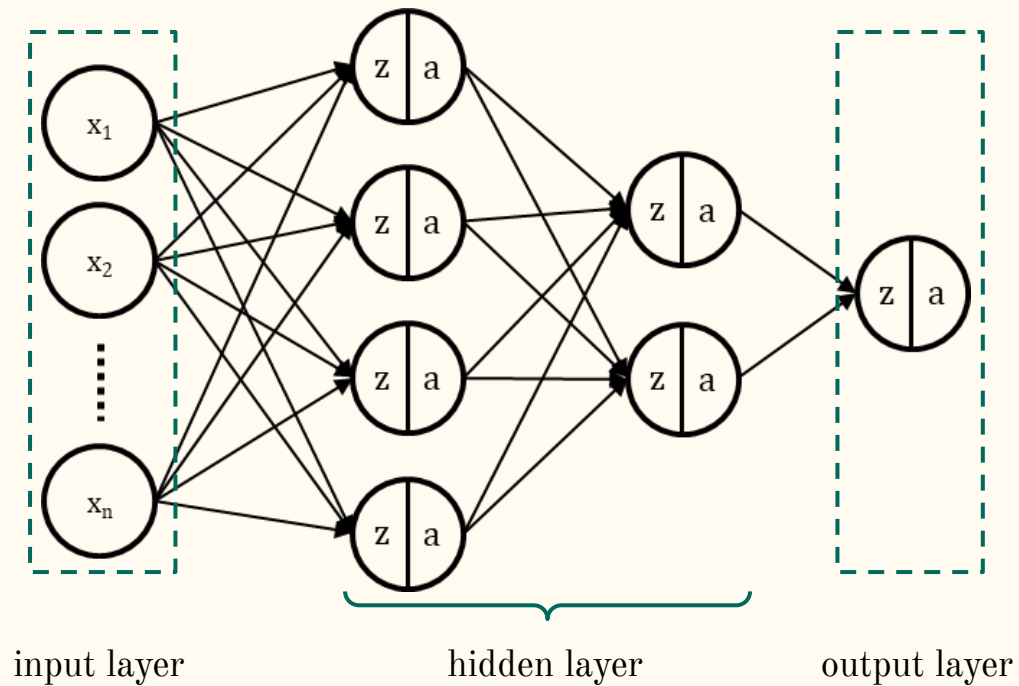
$z|a$: 過 Activation

$a = g(z)$
Activation Function.

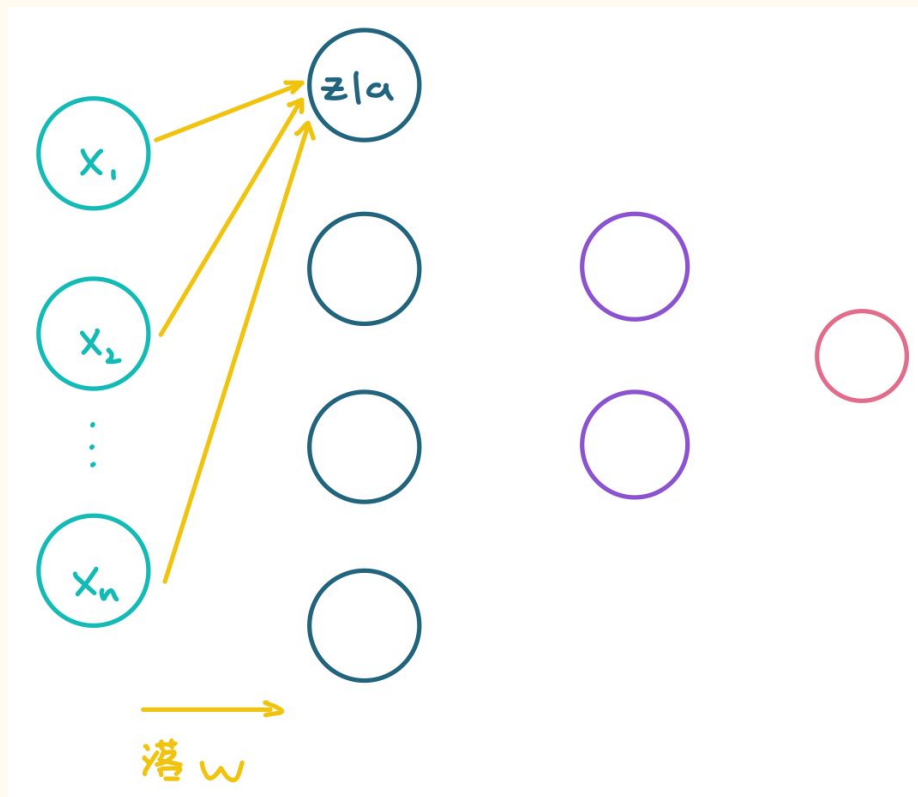
$\hat{y}$: output / prediction



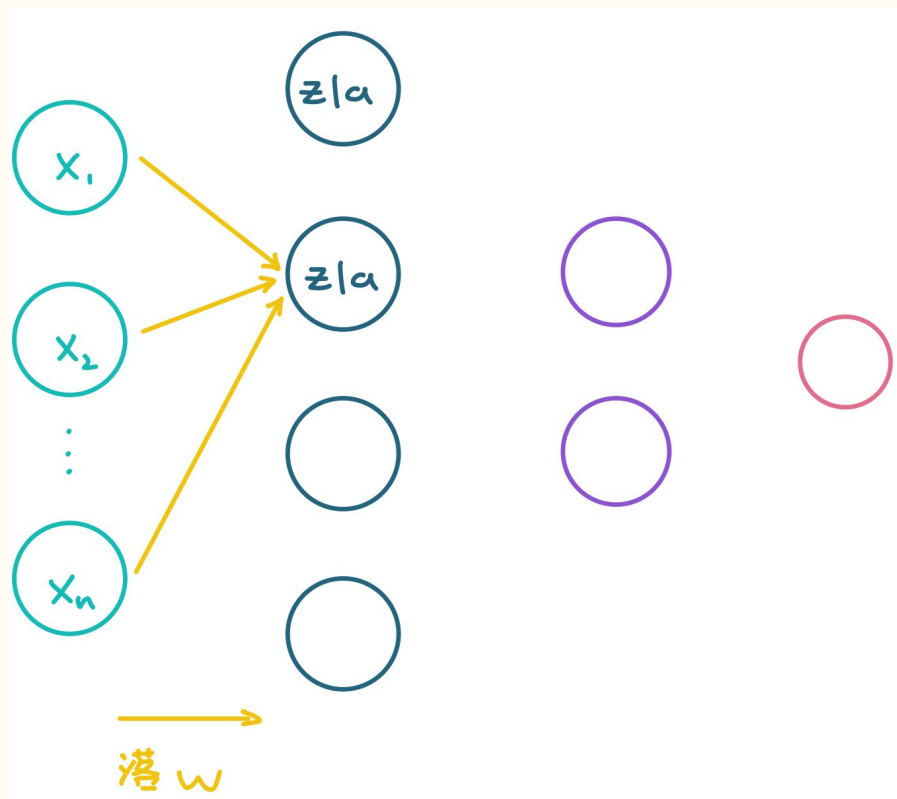
Forward Pass



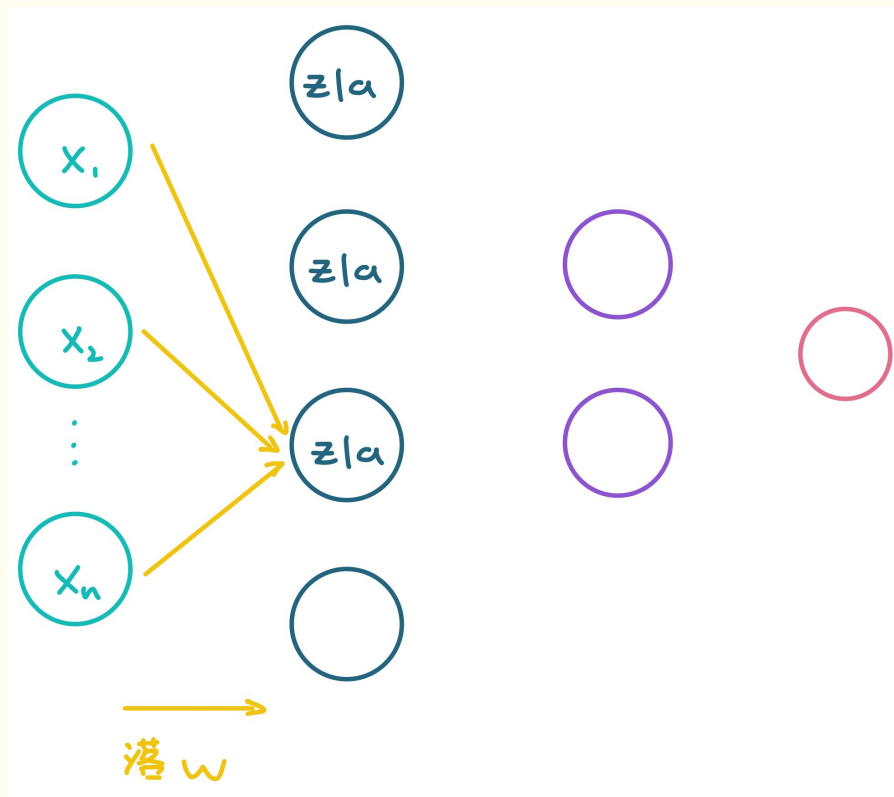
Forward Pass



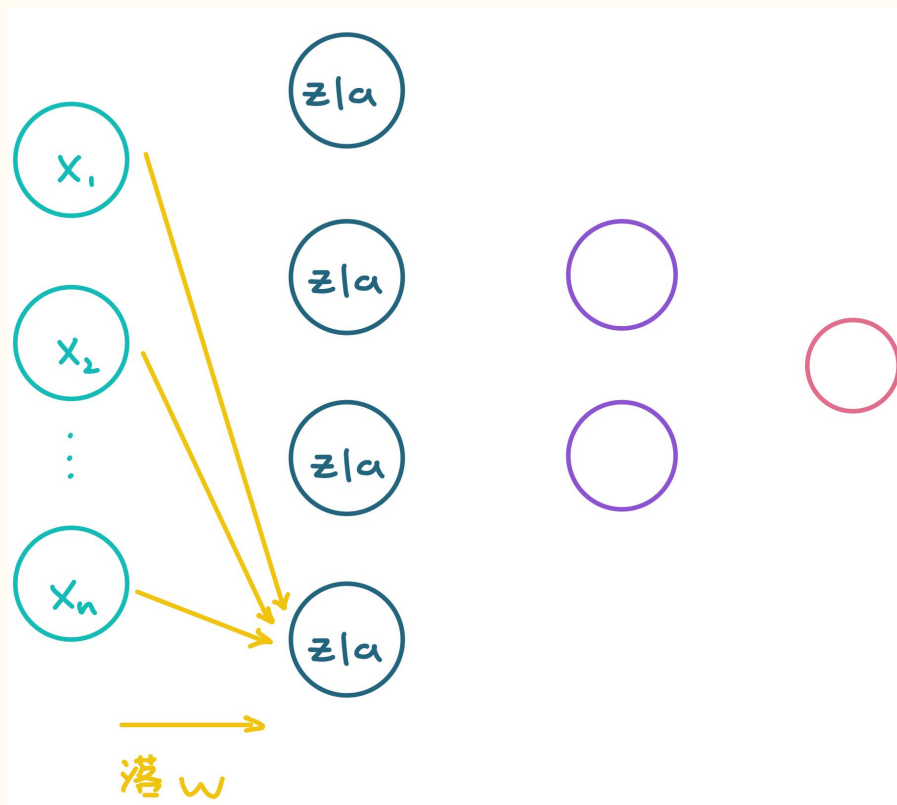
Forward Pass



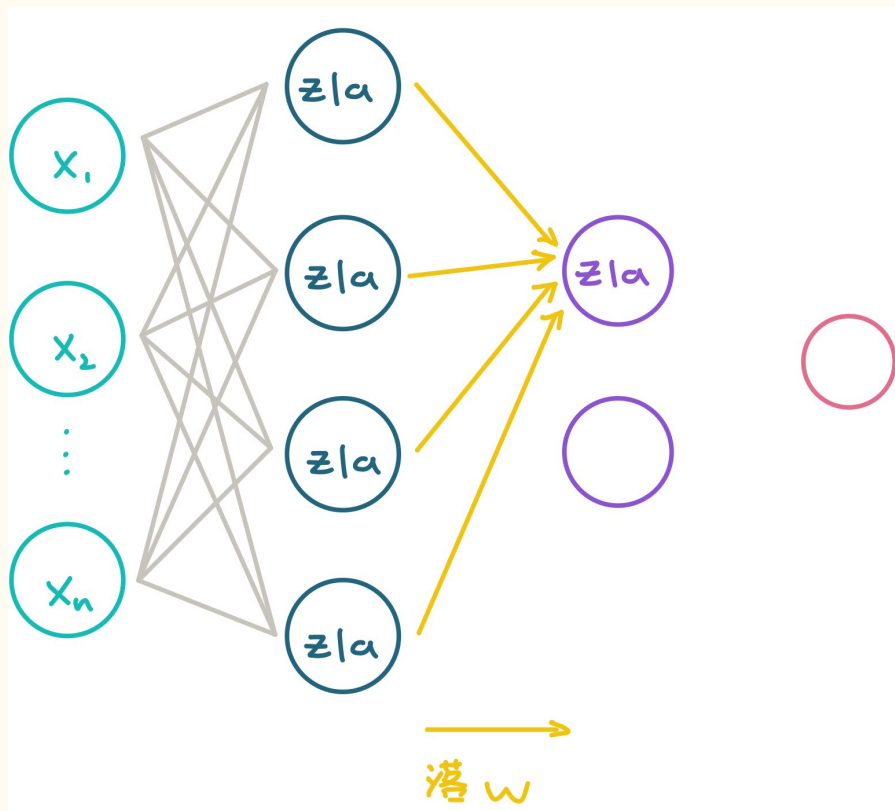
Forward Pass



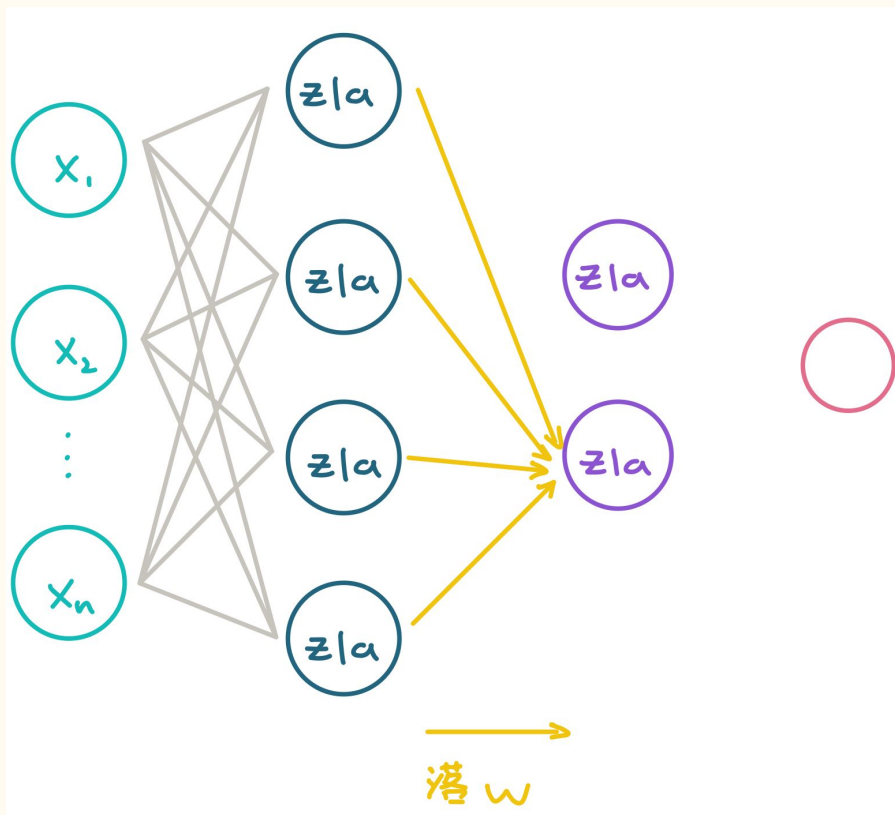
Forward Pass



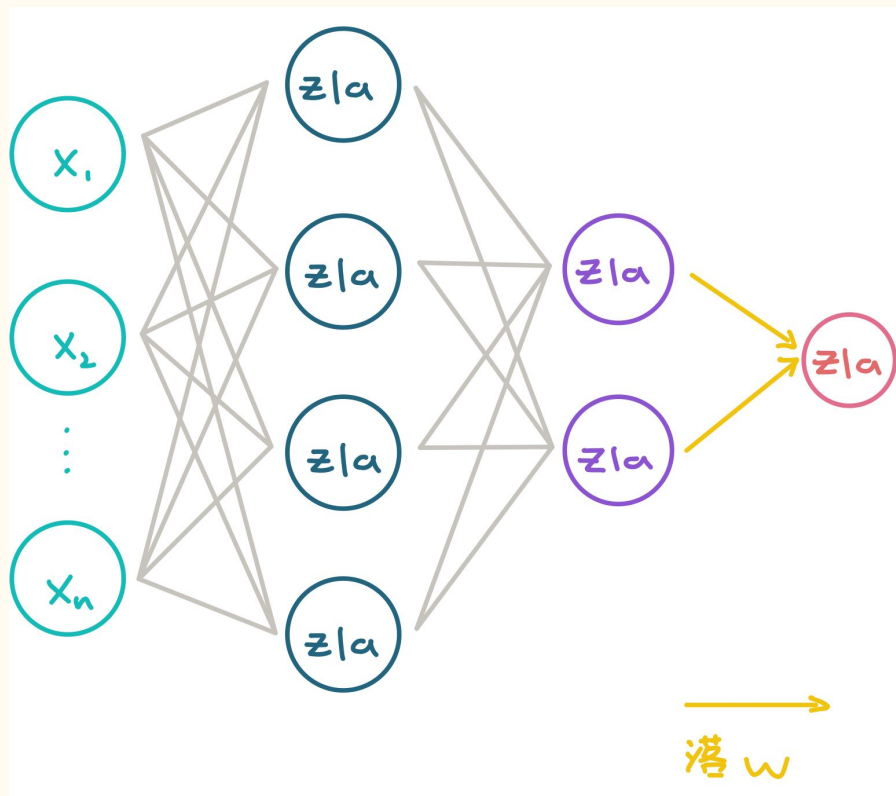
Forward Pass



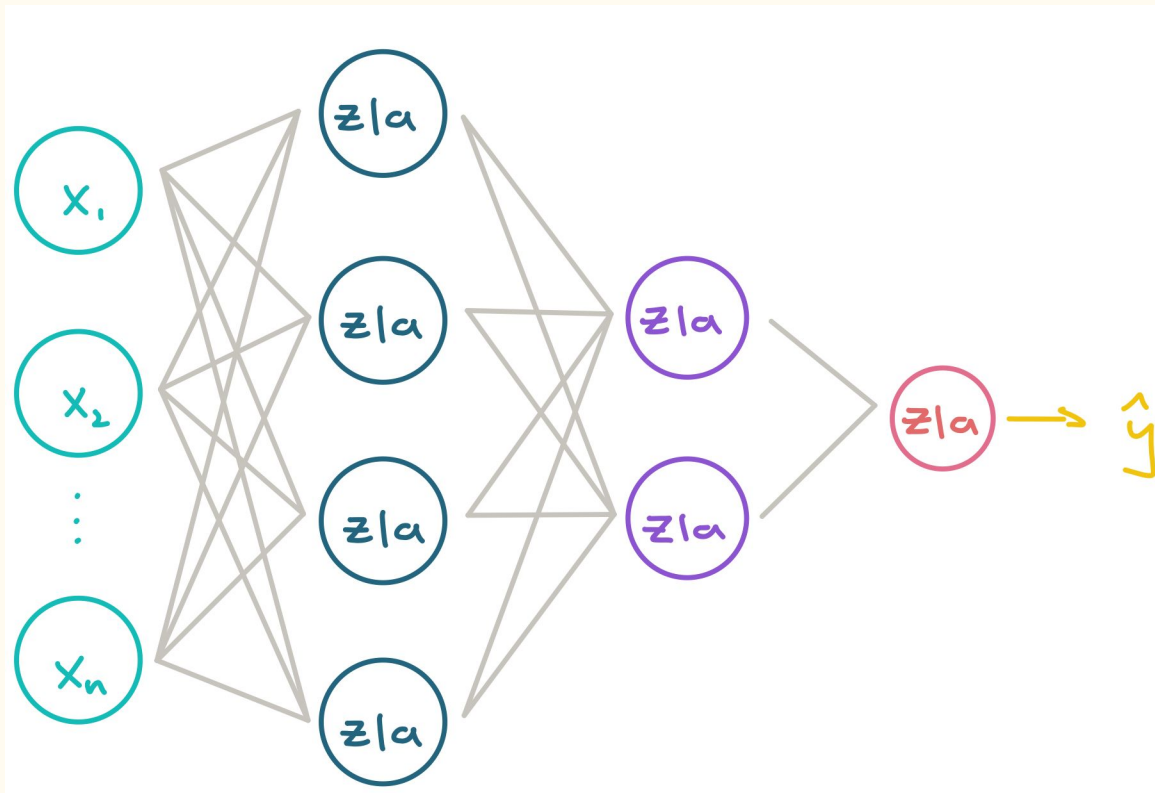
Forward Pass



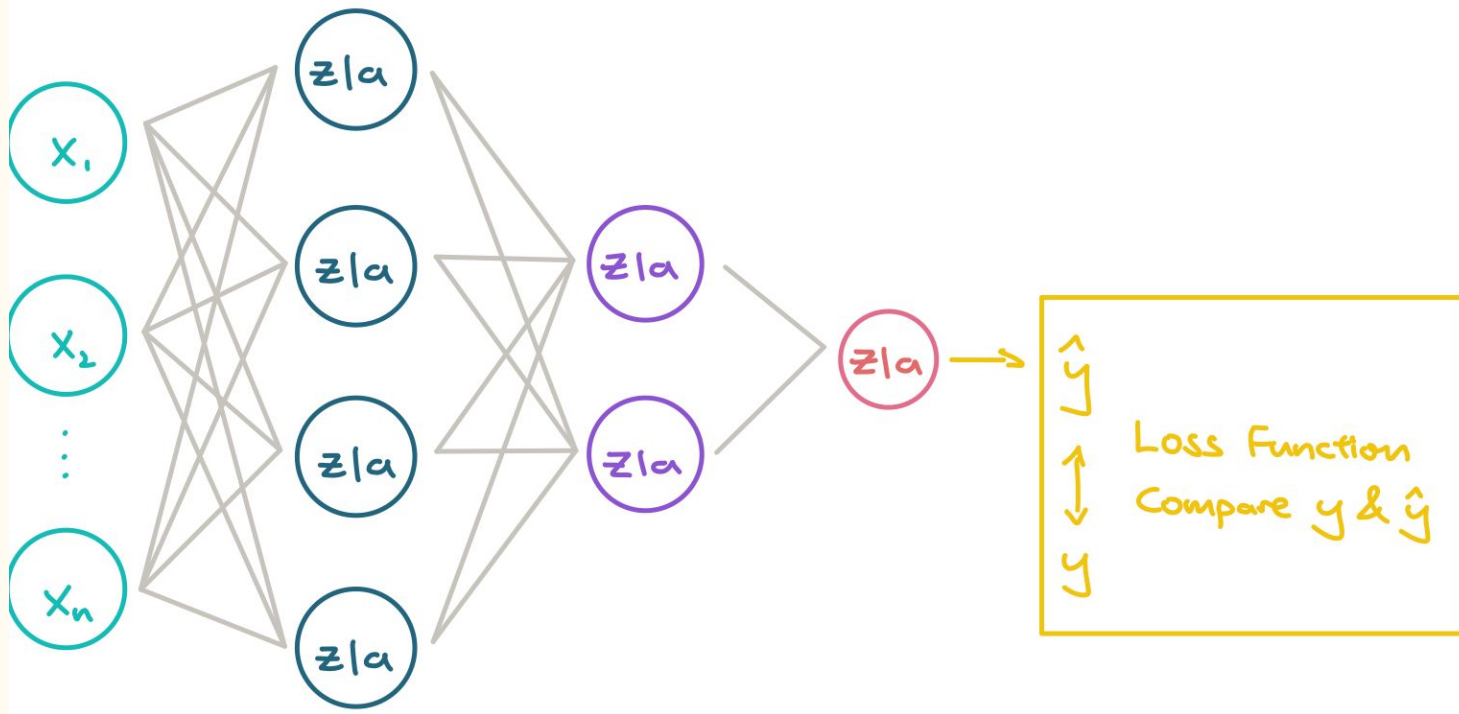
Forward Pass



Forward Pass



Loss Function



Choices of Loss Function

Loss Function for discrete output [Classification] :

Cross - entropy

KL divergence loss

Loss Function for continuous output [Regression] :

MSE loss (L2 regularization)

MAE loss (L1 regularization)

Cross - entropy

This function comes from information theory where the goal is to **measure the difference between two averages of the number of bits of distribution of information.**

$$L(\hat{y}, y) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

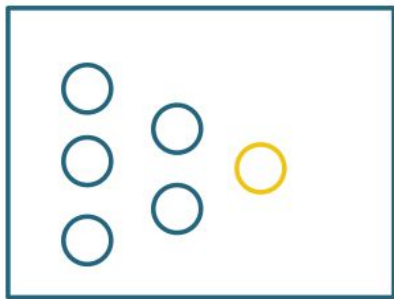
After all, what is **entropy*** ?

*Remark : refer to information or Shannon entropy instead of the entropy in thermodynamics or statistical mechanics, though they share the same idea, bruh...

Information entropy

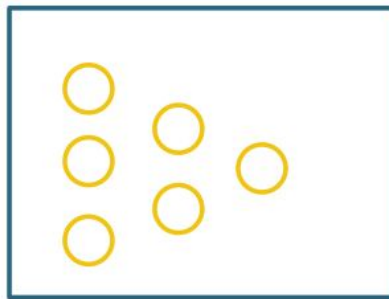
Does infinity surprise make sense ?
Definitely no : how come zero probability
comes with infinity surprise ???

The story starts with 'surprise' and probability...



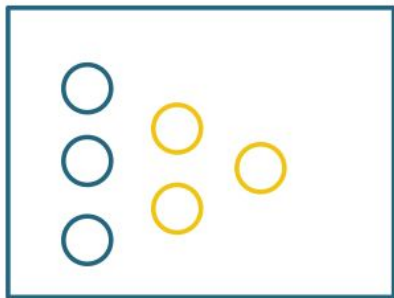
$$P(\text{yellow}) = \frac{1}{6}$$

$$\text{"Surprise"} = \frac{1}{p} = 6$$



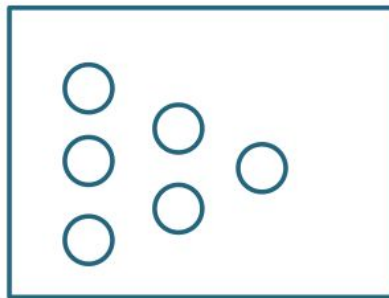
$$P(\text{yellow}) = \frac{1}{2}$$

$$\text{"Surprise"} = \frac{1}{p} = 2$$



$$P(\text{yellow}) = \frac{1}{2}$$

$$\text{"Surprise"} = \frac{1}{p} = 2$$



$$P(\text{yellow}) = \frac{1}{6}$$

$$\text{"Surprise"} = \frac{1}{p} = 6$$

Information Entropy

Therefore we define : [log refers to base 2 for information bits 0 and 1]

$$\text{Surprise} = \log\left(\frac{1}{p}\right)$$

$$= -\log(p)$$

Entropy = Expectation of surprise

$$= \sum_i p_i \log\left(\frac{1}{p_i}\right)$$

$$= -\sum_i p_i \log(p_i)$$

Recall :

$$\langle \text{Continuous} \rangle E(X) = \int_{-\infty}^{\infty} x p(x) dx$$

↖ Pdf

$$\langle \text{Discrete} \rangle E(X) = \sum_i x_i f(x_i)$$

↖ pmf

Information entropy

1. For probability ranges from 0 to 1, $-\log(p)$ / surprise must be positive
2. For event that must be true ($p = 1$), $-\log(p)$ / surprise must be 0
3. As the probability of the event gets smaller, $-\log(p)$ / surprise gets larger

[The event carries more information when probability gets smaller]

4. For multiple independent events, we multiply the probabilities, so $-\log(p)$ / surprise adds up in this case

$$H(x) = -E_{x \sim p}[\log_2 p(x)] = - \sum_i p_i \log_2 p_i$$

Shannon Entropy : short example

Assume $P(A) = 0.5$, $P(B) = 0.25$, $P(C) = 0.25$

$$H = -\frac{1}{2}\log_2\left(\frac{1}{2}\right) - \frac{1}{4}\log_2\left(\frac{1}{4}\right) - \frac{1}{4}\log_2\left(\frac{1}{4}\right) = 0.5 + 0.5 + 0.5 = 1.5$$

A : 0 1 $\Rightarrow$ 1 bit

B : 00 01 $\Rightarrow$ 2 bits
10 11

C : 00 01 $\Rightarrow$ 2 bits
10 11

Assume perform trial 200 times

$$\frac{100 \times 1 + 50 \times 2 + 50 \times 2}{200} = 1.5$$



$$\boxed{\frac{100}{200}} \times 1 + \boxed{\frac{50}{200}} \times 2 + \boxed{\frac{50}{200}} \times 2 = 1.5$$

Conclusion of Shannon Entropy

The Noiseless Coding Theorem (Shannon, 1948):

The entropy is a lower bound on the number of bits needed to transmit the state of a random variable (r.v.)

Remarks :

Always remember that when we are discussing under the context of Machine Learning, logarithm refers to natural log

So the base 2 logarithm is converted to natural log when we shift the discussion to ML

$$H(p) = E_{x \sim p}[-\ln p(x)]$$

From information entropy to cross-entropy

Assume binary classification (bernoulli trial):

$$H(p, q) = -p_{positive} \cdot \ln(q_{positive}) - p_{negative} \cdot \ln(q_{negative})$$

Binary clf : only 2 states (0 or 1)

$$H(p, q) = -p_{positive} \cdot \ln(q_{positive}) - (1 - p_{positive}) \cdot \ln(1 - q_{positive})$$

From information entropy to cross-entropy

p is actual distribution [1 : positive | 0 : negative]

q is model prediction [1 : positive | 0 : negative]

p_positive $\rightarrow$ y

q_positive $\rightarrow$ q

$$H(p, q) = -p_{positive} \cdot \ln(q_{positive}) - (1 - p_{positive}) \cdot \ln(1 - q_{positive})$$

$$H(p, q) = -y \cdot \ln(q) - (1 - y) \cdot \ln(1 - q)$$

KL divergence (Kullback-Leibler Divergence)

When 2 distributions corresponds the exact same input random variable X

KL divergence measures the difference between the two distributions.

Cross-entropy offers a way to quantify the average number of bits (amount of information) required to encode data following distribution P, using distribution Q.

original bits (P) + extra bits (KL div) = total bits (Q)

$$D_{KL}(q||p) = - E_{x \sim q}[\ln p(x) - \ln q(x)] = H(q, p) - H(q)$$

KL divergence

$$D_{KL}(p||q) = -E_{x \sim p}[\ln q(x) - \ln p(x)] = -E_{x \sim p}[\ln \frac{q(x)}{p(x)}]$$

$$D_{KL}(p||q) = E_{x \sim p}[-\ln q(x)] - E_{x \sim p}[-\ln p(x)] = H(p, q) - H(p)$$

KL divergence is actually the information / uncertainty added by the model

Aim : minimise the cross-entropy loss of model

Summary : Information theory and Cross-entropy

Entropy : information carried by original actual distribution

$$\text{Entropy: } H(p) = E_{x \sim p}[-\ln p(x)]$$

Cross-entropy : total information carried by prediction of model

$$H(p, q) = E_{x \sim p}[-\ln q(x)]$$

KL divergence : extra information/uncertainty added by model

$$D_{KL}(p||q) = H(p, q) - H(p)$$

Entropy + KL divergence = Cross-entropy

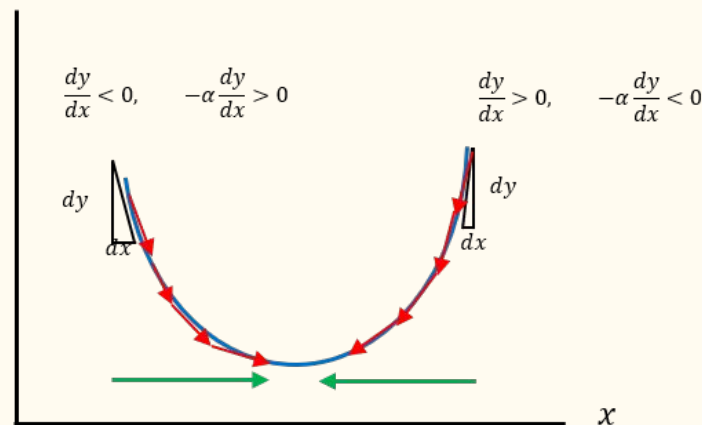
Recap : Gradient Descent

Loss function is the difference between predicted class and actual class.

Aim : adjust the weight and bias until the loss function is **minimum**.

Method : Take gradient of loss function $\rightarrow$ set it to 0 $\rightarrow$ tune parameter b, w
 $y = f(x)$

1. Take Gradient of the Loss Function
2. Pick random values for the parameters
3. Plug the parameters value into the gradient
4. Step sizes : step size = slope x learning rate
5. New parameters = Old parameter - step size



Recap : Gradient Descent

Aim : adjust the weight and bias until the loss function is **minimum**.

Method : Take gradient of loss function $\rightarrow$ set it to 0 $\rightarrow$ tune parameter b, w

Note that : Prediction / output is a function that includes all of the parameters all the way from input to output.

$$L(\hat{y}, y) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

$$L(\hat{y}, y) = L(x, w^{[1]}, b^{[1]}, w^{[2]}, b^{[2]}, w^{[3]}, b^{[3]}, y)$$

Key : Gradient of Loss function = chain rule of all derivatives of parameters

Steps : Take gradient of L $\rightarrow$ chain rule $\rightarrow$ update b, w in backward direction

Gradient descent

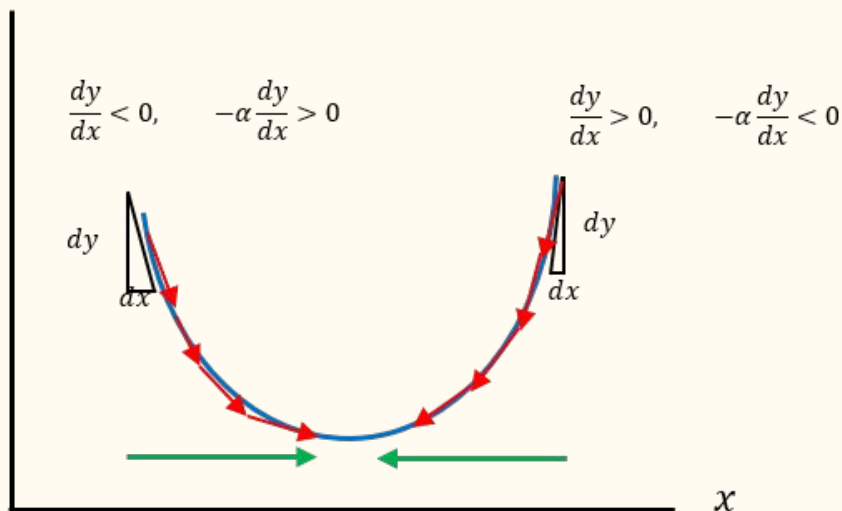
$$L(\hat{y}, y) = L(x, w^{[1]}, b^{[1]}, w^{[2]}, b^{[2]}, w^{[3]}, b^{[3]}, y)$$

$$w^{[l]} := w^{[l]} - \alpha \frac{\partial L}{\partial w^{[l]}}$$

$$b^{[l]} := b^{[l]} - \alpha \frac{\partial L}{\partial b^{[l]}}$$

alpha : learning rate

$$y = f(x)$$



Forward Pass

$$z_1^{[1]} = w_{11}^{[1]} a_1^{[0]} + w_{12}^{[1]} a_2^{[0]} + \dots + w_{1n}^{[1]} a_n^{[0]} + b_1^{[1]}$$

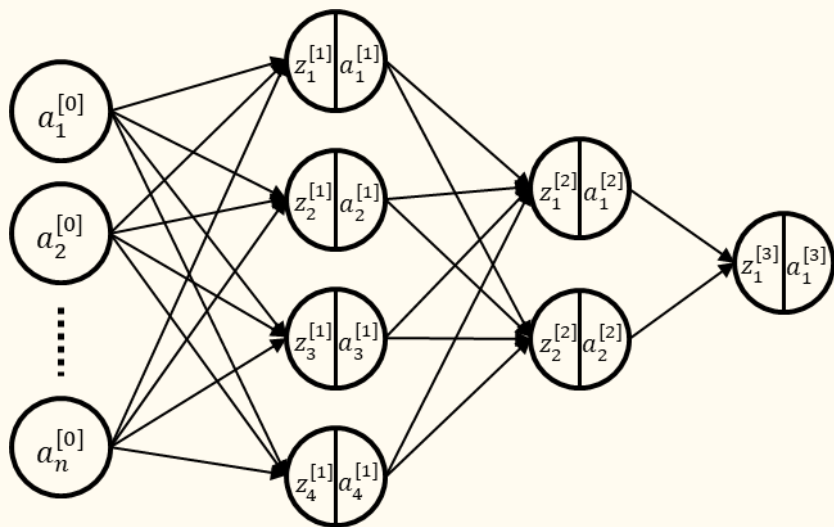
$$a_1^{[1]} = g(z_1^{[1]})$$

$$z_2^{[1]} = w_{21}^{[1]} a_1^{[0]} + w_{22}^{[1]} a_2^{[0]} + \dots + w_{2n}^{[1]} a_n^{[0]} + b_2^{[1]}$$

$$a_1^{[2]} = w_{11}^{[2]} a_1^{[1]} + w_{12}^{[2]} a_2^{[1]} + w_{13}^{[2]} a_3^{[1]} + w_{14}^{[2]} a_4^{[1]} + b_1^{[2]}$$

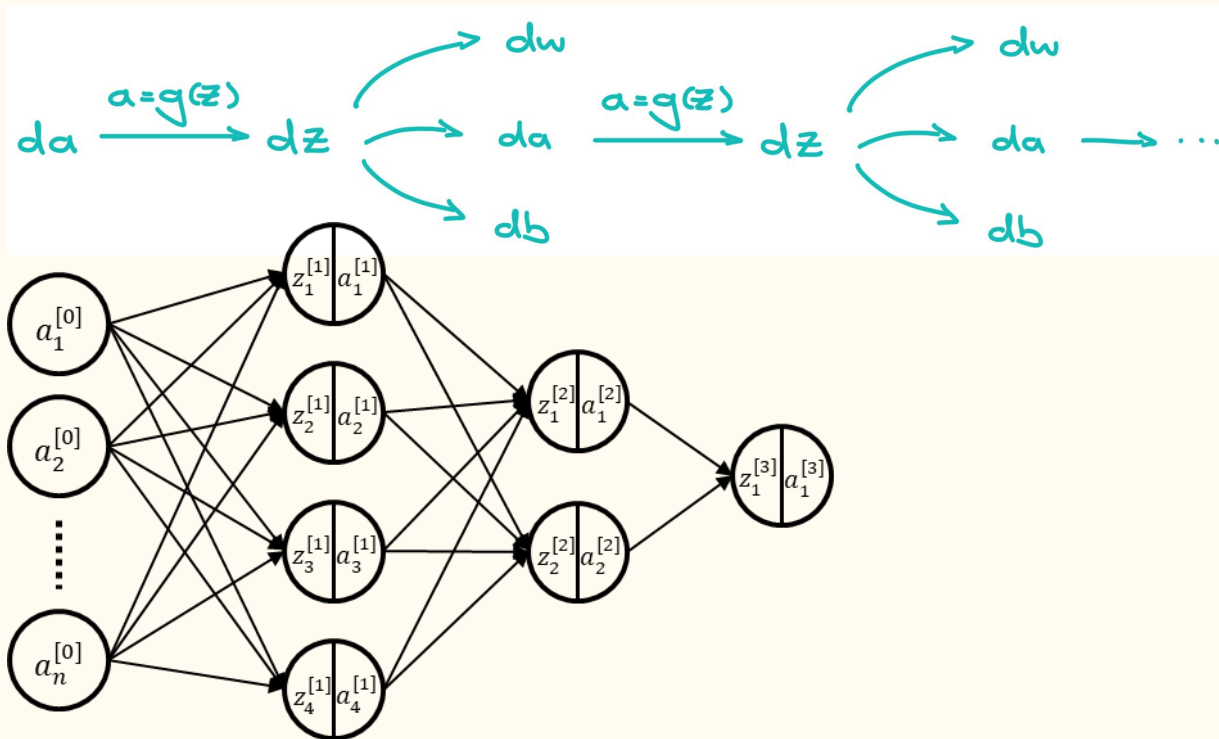
$$z_{11}^{[3]} = w_{11}^{[3]} a_1^{[2]} + w_{21}^{[3]} a_2^{[2]} + b_1^{[3]}$$

$$a_1^{[3]} = g(z_{11}^{[3]}) = \hat{y}$$



Back propagation big picture

We will neglect the numerator of partial derivatives for simplicity



Back propagation big picture

We will neglect the numerator of partial derivatives for simplicity



Back propagation (chain rule)

$$\frac{\partial L}{\partial \hat{y}} = d\hat{y} = da^{[3]} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = -\frac{y}{a^{[3]}} + \frac{1-y}{1-a^{[3]}} = -\frac{y}{g(z^{[3]})} + \frac{1-y}{1-g(z^{[3]})}$$

$$g(z) = \text{sigmoid}(z) = \frac{1}{1+e^{-z}}$$

$$g'(z) = \frac{-1}{(1+e^{-z})^2} \times (-e^{-z}) = \frac{1+e^{-z}-1}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} - \frac{1^2}{(1+e^{-z})^2} = g(z) - g(z)^2 = g(z)(1-g(z))$$

$$\frac{\partial L}{\partial z^{[3]}} = dz^{[3]} = da^{[3]} \frac{da^{[3]}}{dz^{[3]}} (\text{chain rule}) = da^{[3]} g'(z^{[3]})$$

$$= -y(1-g(z^{[3]})) + (1-y)g(z^{[3]}) = -y + yg(z^{[3]}) + g(z^{[3]}) - yg(z^{[3]}) = g(z^{[3]}) - y$$

BP chain rule

$$\frac{\partial L}{\partial w^{[3]}} = dw^{[3]} = \frac{\partial L}{\partial z^{[3]}} \frac{dz^{[3]}}{dw^{[3]}} = dz^{[3]} \times a^{[2]} (\text{chain rule} + \text{partial derivative})$$

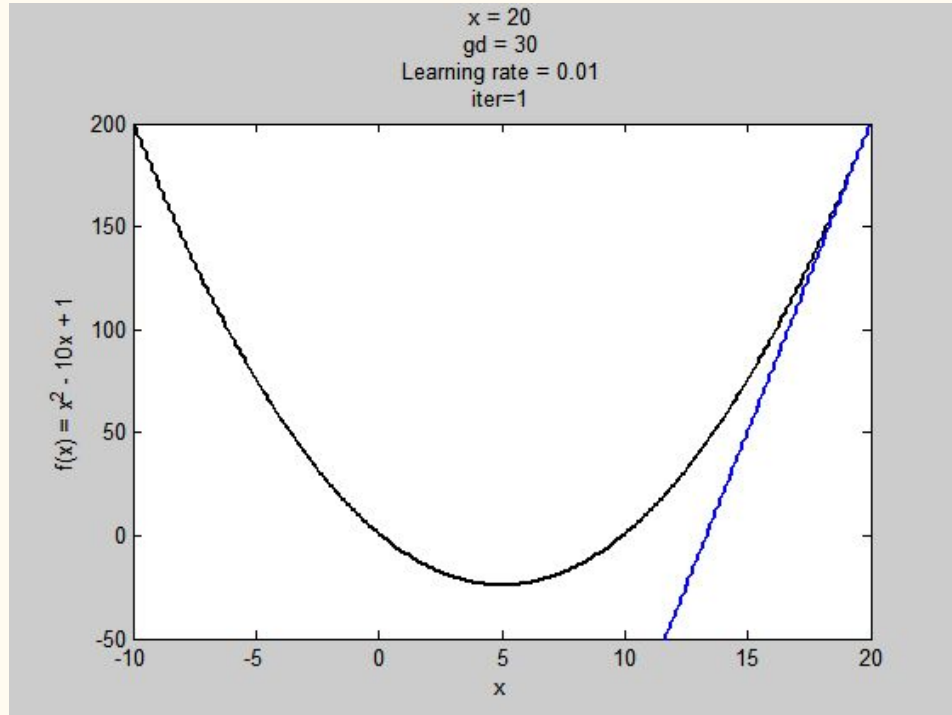
$$\frac{\partial L}{\partial b^{[3]}} = db^{[3]} \frac{\partial L}{\partial z^{[3]}} \frac{dz^{[3]}}{db^{[3]}} = dz^{[3]}$$

$$\frac{\partial L}{\partial a^{[2]}} = da^{[2]} \frac{\partial L}{\partial z^{[3]}} \frac{dz^{[3]}}{da^{[3]}} = dz^{[3]} \times w^{[3]}$$

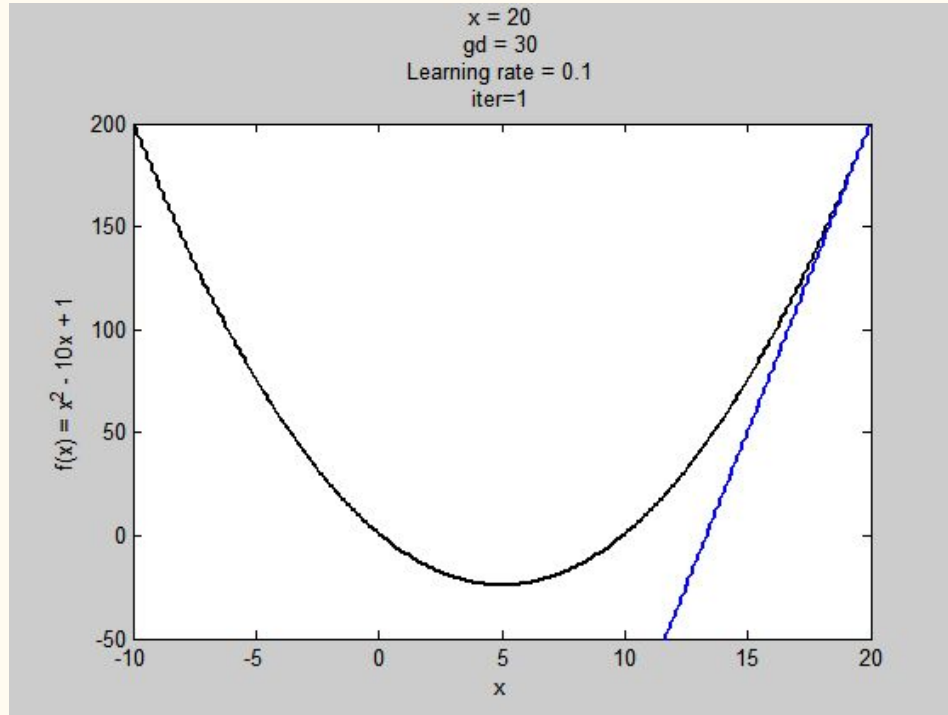
z is a function of w, a, b

so we take partial derivative respectively

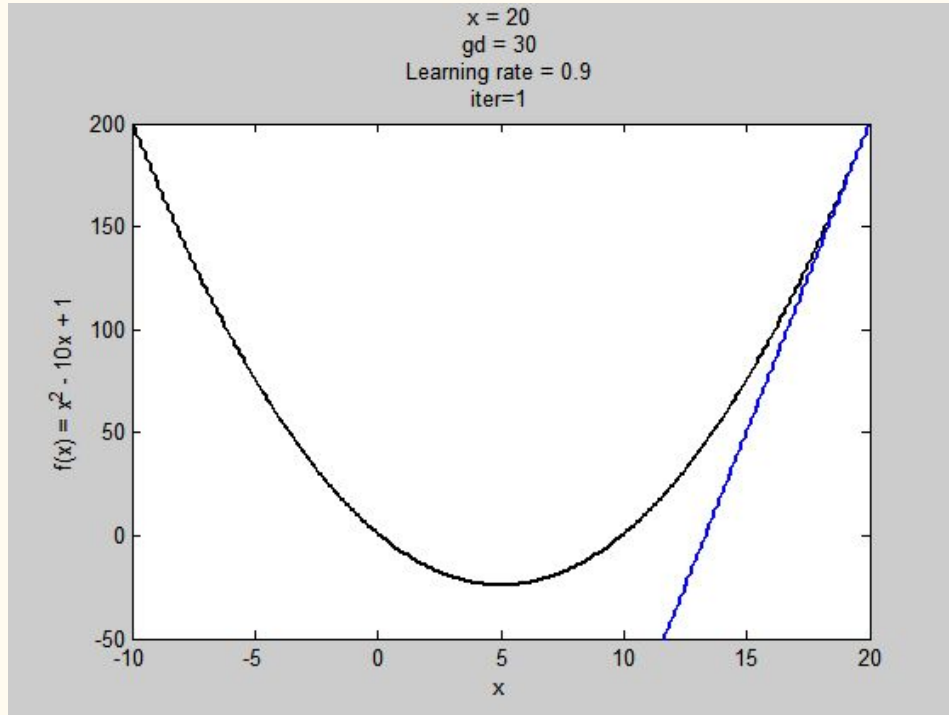
BP visualization



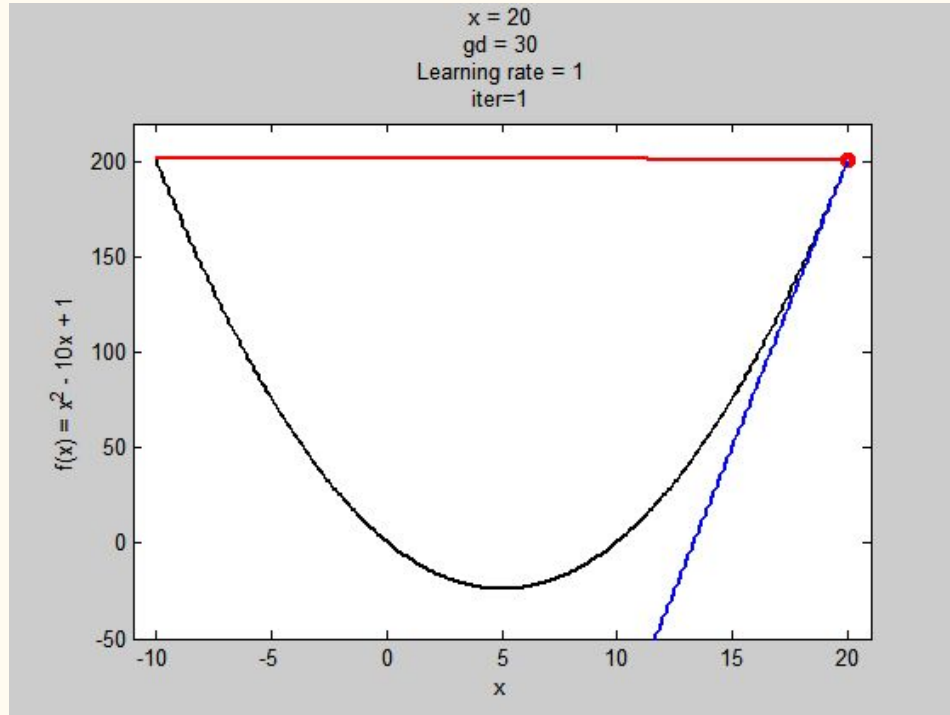
BP visualization



BP visualization



BP visualization



Stochastic Gradient Descent

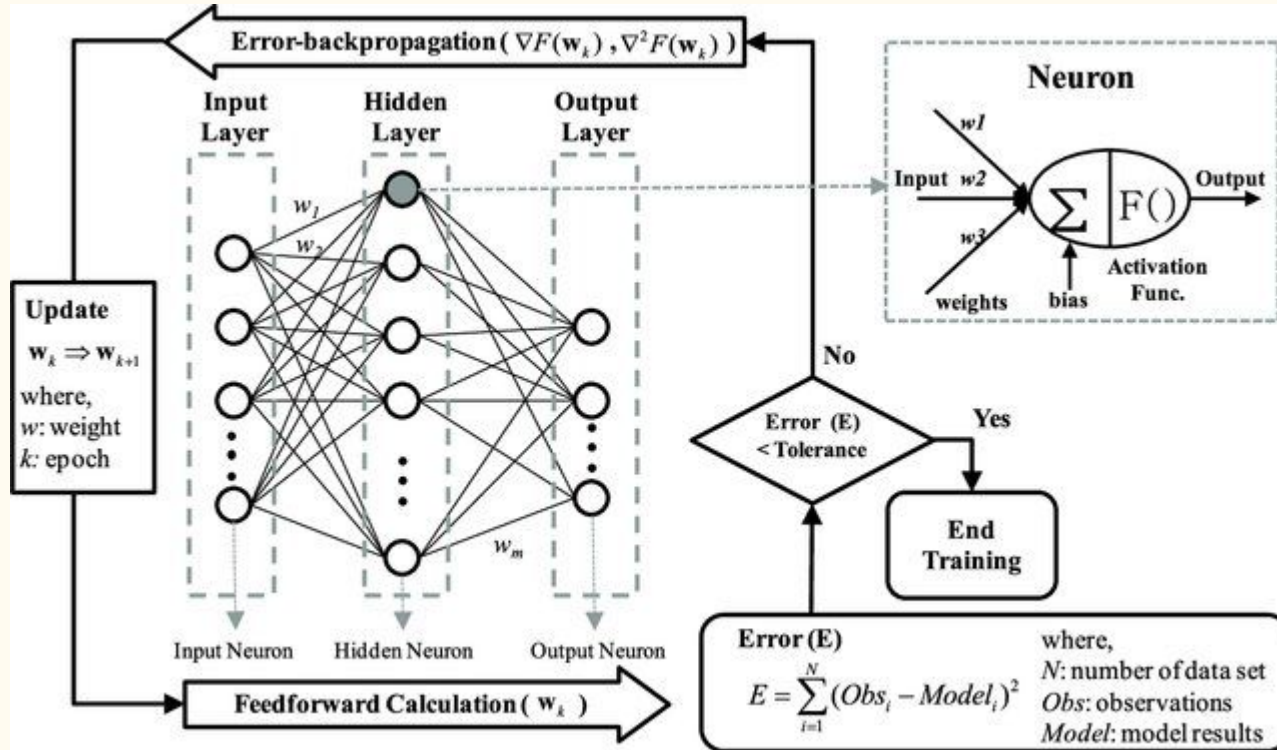
Stochastic = Random

Since GD starts with a value and converge to the local minimum, the choice of value affects the results when there are multiple local minimum.

But do we need to compute GD with all values to search for global minimum ?

We can choose a random value each time, such that the randomness is represented as stochastic gradient descent to reduce computation.

Back Propagation big picture



Choices of Activation Function

1. Sigmoid
2. Softmax
3. Tanh
4. ReLU
5. Leaky ReLU

Key : We will use derivatives and monotonicity to understand these functions

Derivatives and Monotonicity

Derivatives

change in y with respect to x (slope gradient)

Monotonicity

A function is monotonic if it is non-increasing or non-decreasing

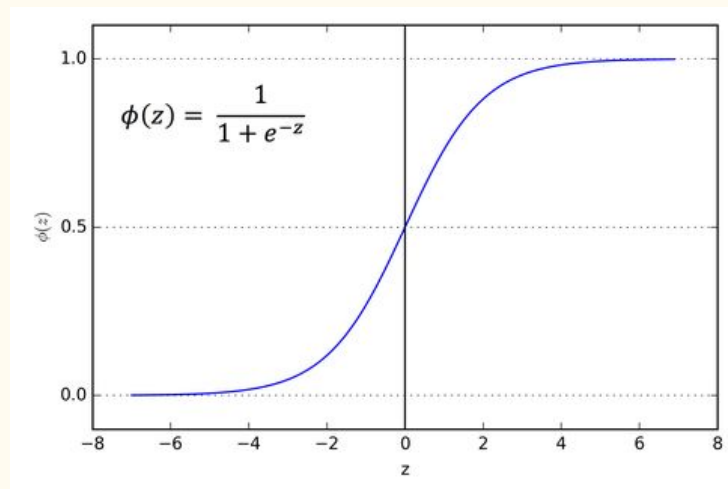
Sigmoid (Logistic) Activation Function

Range : 0 to 1

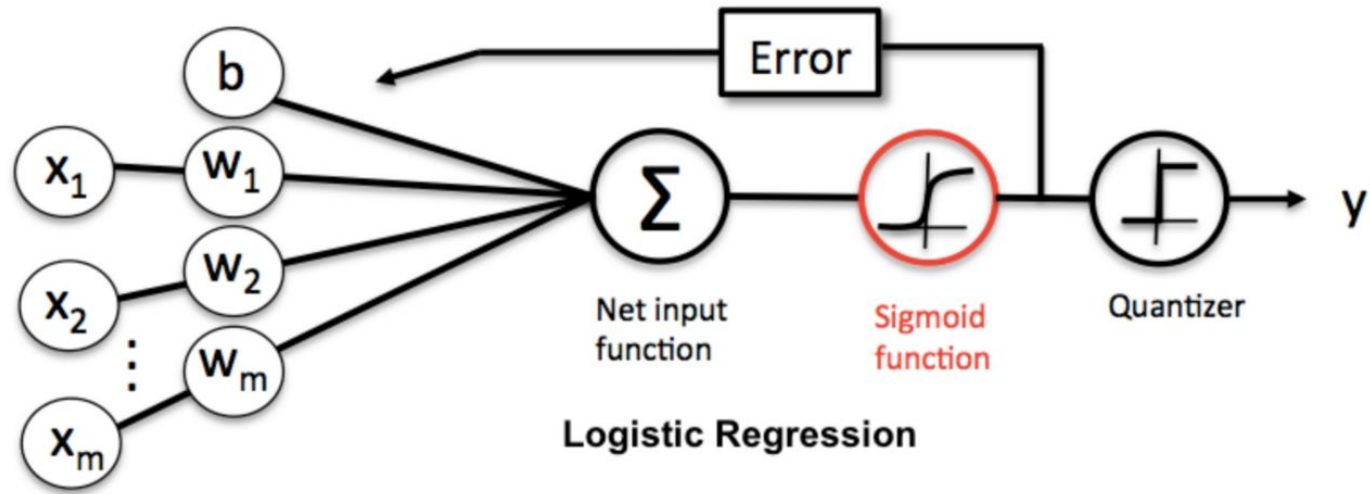
Predict probability as output

→ Application : binary classification output layer

- differentiable
- monotonic
- derivative is not monotonic



How back propagation works with sigmoid



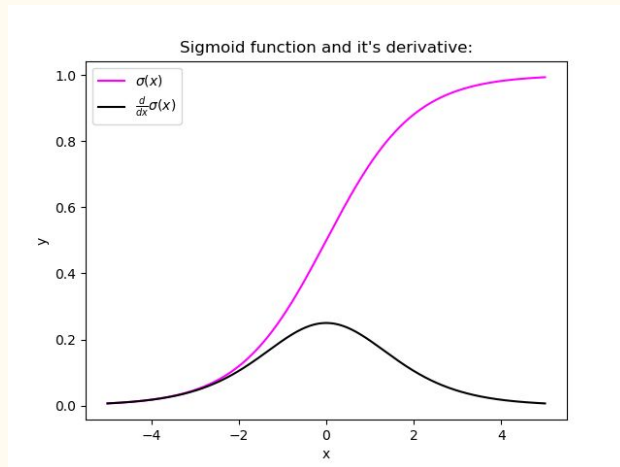
Gradient exploding and vanishing

Gradient vanishing : gradient getting smaller and smaller in BP $\rightarrow$ tends to 0

Gradient exploding : gradient getting greater and greater in BP $\rightarrow$ weight diverges

Why Gradient Exploding / Vanishing ?

- ❖ for larger inputs (negative or positive), it saturates at 0 or 1 with a derivative very close to zero
- ❖ when the backpropagation algorithm chips in, it virtually has no gradients to propagate backward in the network
- ❖ little residual gradients exist keeps on diluting as the algorithm progresses down through the top layers
- ❖ leaves nothing for the lower layers



Tanh (Hyperbolic tangent) Activation Function

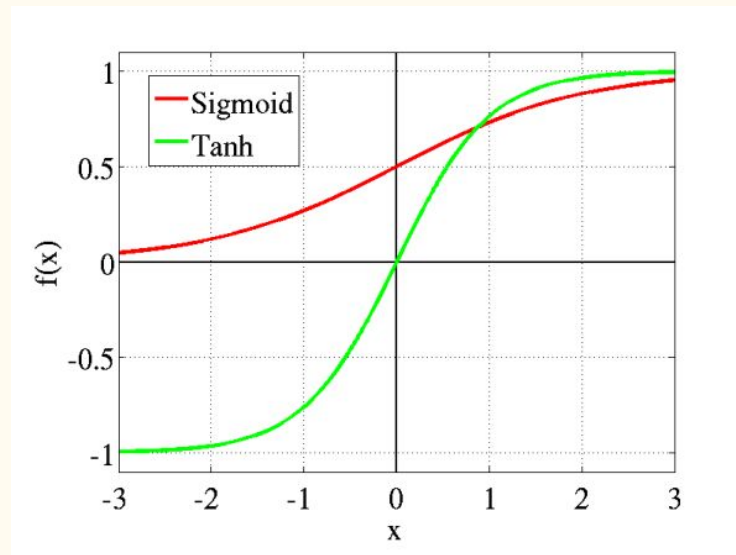
just a better sigmoid which negative value is mapped to strongly negative output.

Range : -1 to 1

Predict probability as output

→ Application : binary classification output layer

- differentiable
- monotonic
- derivative is not monotonic



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Softmax Function

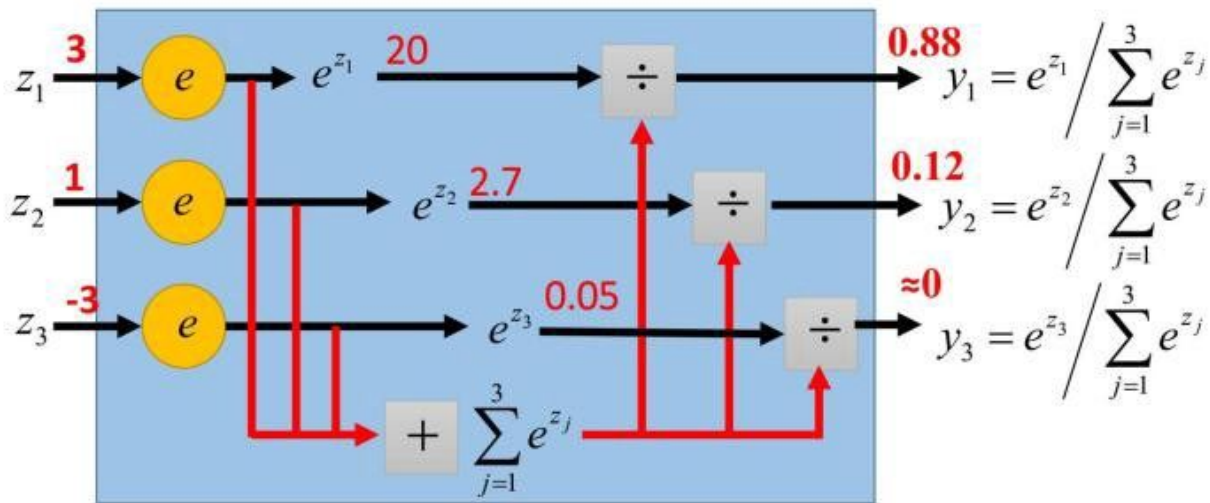
binary clf : sigmoid

multi-class clf : softmax

$$\frac{\exp(x_i)}{\sum_{i=0}^n \exp(x_i)}$$

- Softmax layer as the output layer

Softmax Layer



Probability:

■ $1 > y_i > 0$

■ $\sum_i y_i = 1$

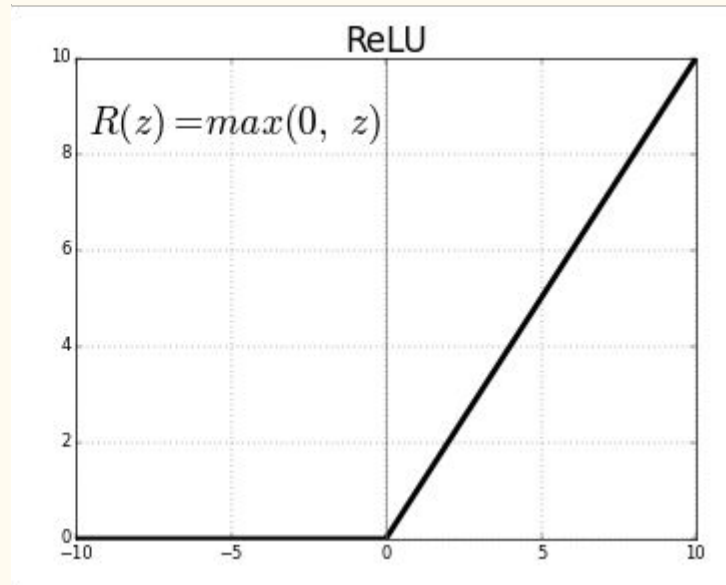
ReLU (Rectified Linear Unit)

ReLU is just linear function, with all negative value being mapped to 0.

Range : 0 to inf

→ application : CNN (most common)

- function and derivatives are monotonic



Optimization

1. Dropout regularization
2. Normalization
3. Mini-batch GD
4. Optimizer

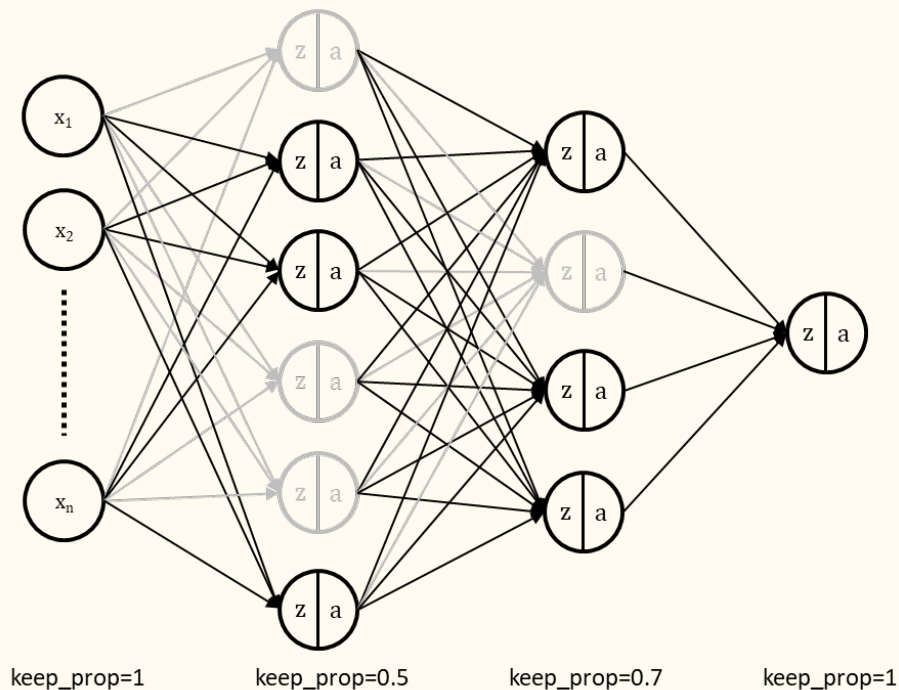
Dropout regularization

Introduce a new hyperparameter p in each iterations

Only keep p (%) node in forward pass to prevent overfitting

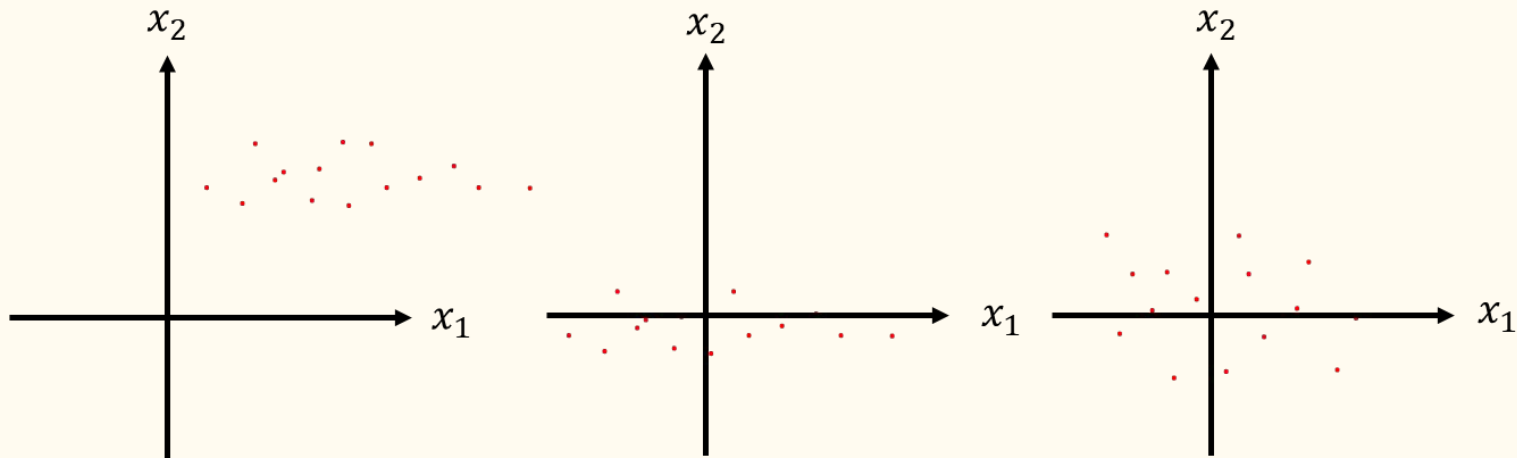
imagine we turn off different neurons each time and we are training different thinner NN each time

→ combine the key properties each time

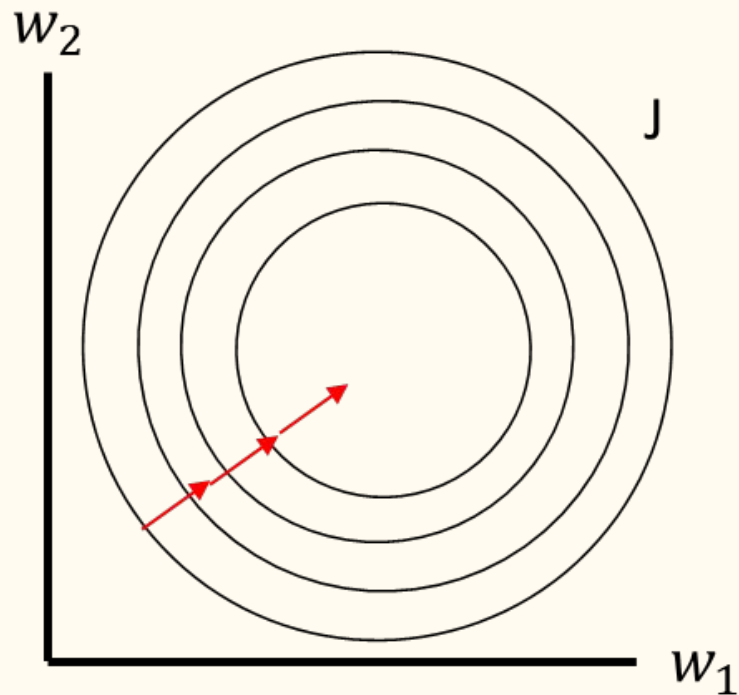
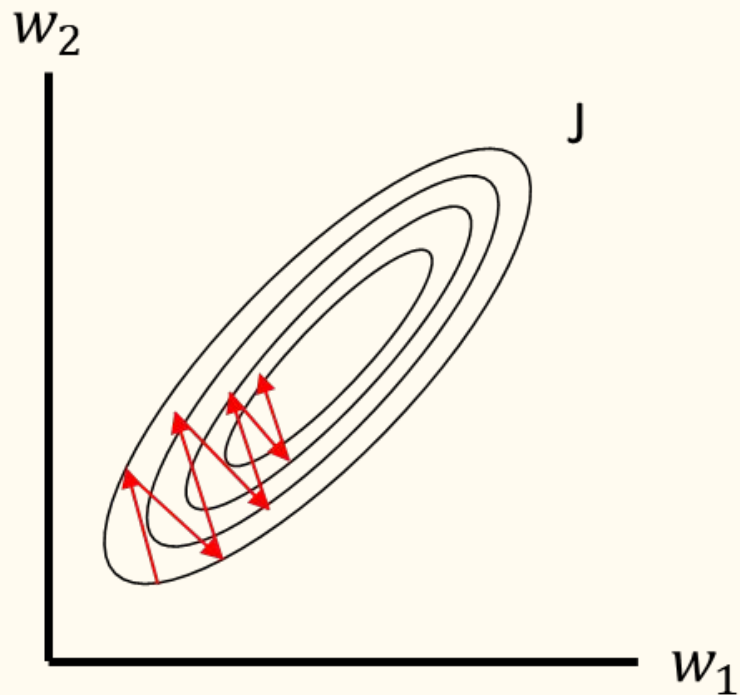


Noramalization

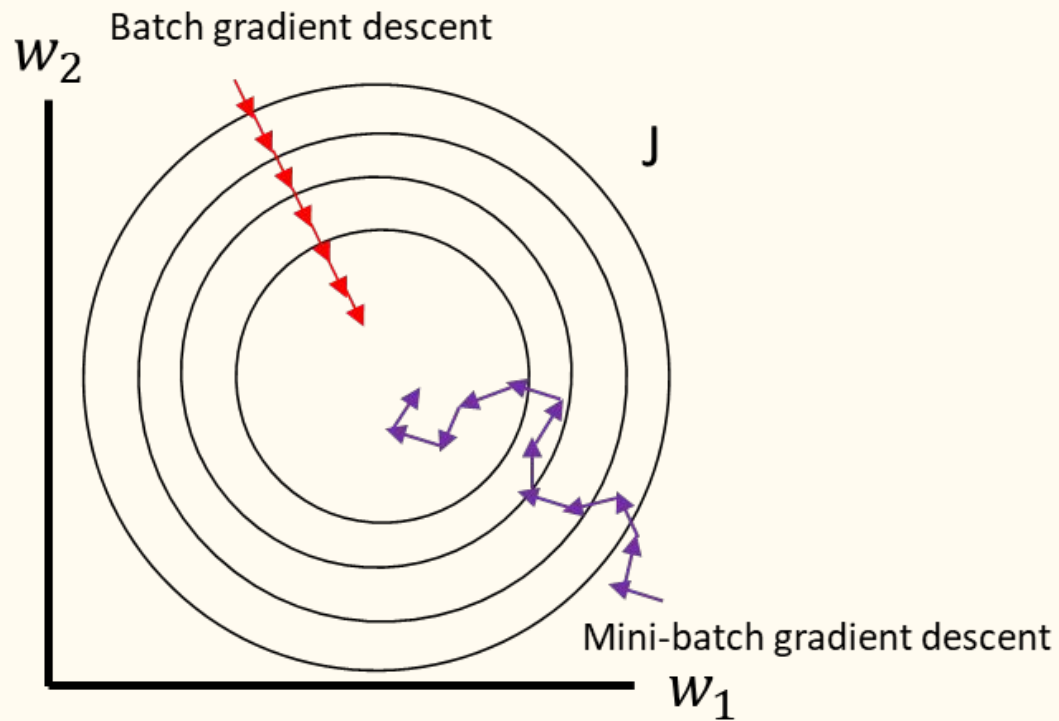
$$\mu = \sum_{i=1}^m x^{(i)}, \sigma^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)^2$$
$$x := \frac{x - \mu}{\sigma}$$



Why normalisation ?



Mini-batch GD



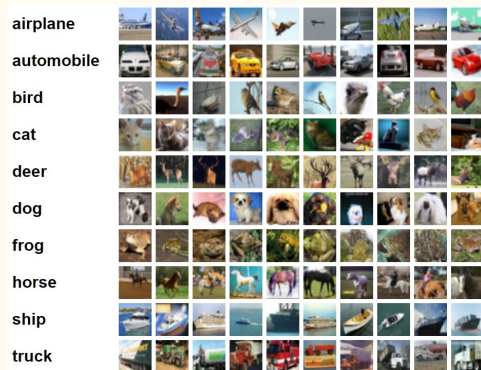
A bit of history...

Why is deep learning so popular?
What is deep learning?

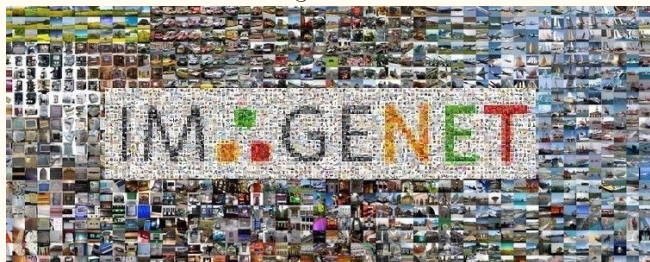
Iris Flower Dataset



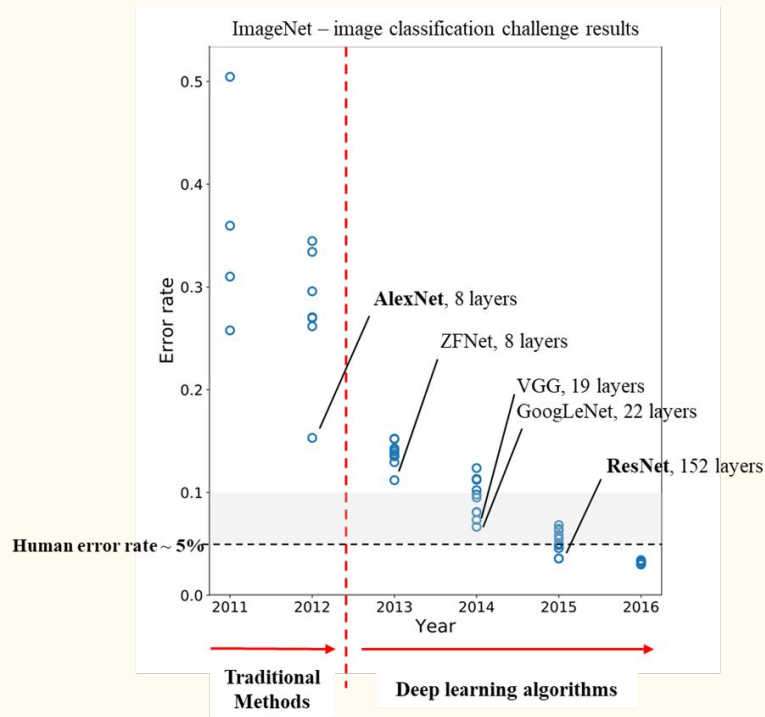
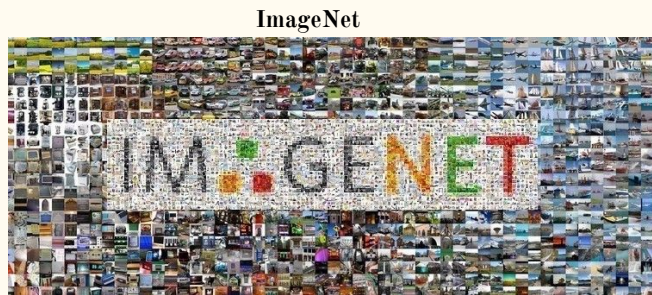
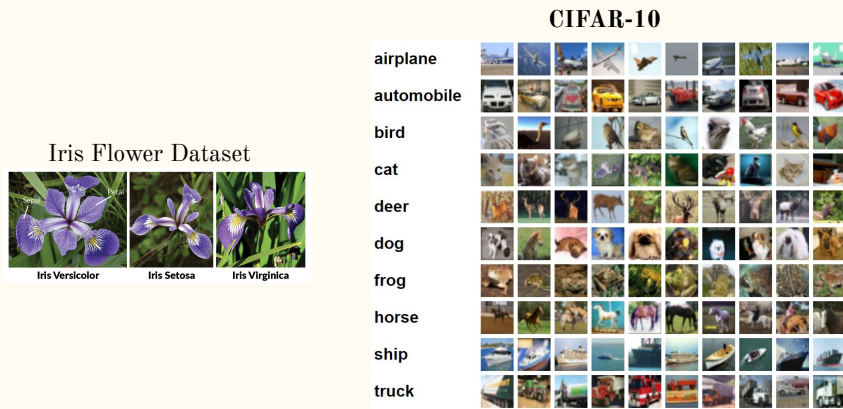
CIFAR-10



ImageNet



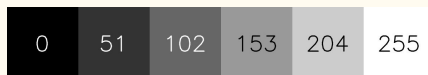
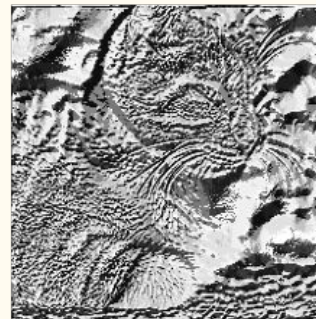
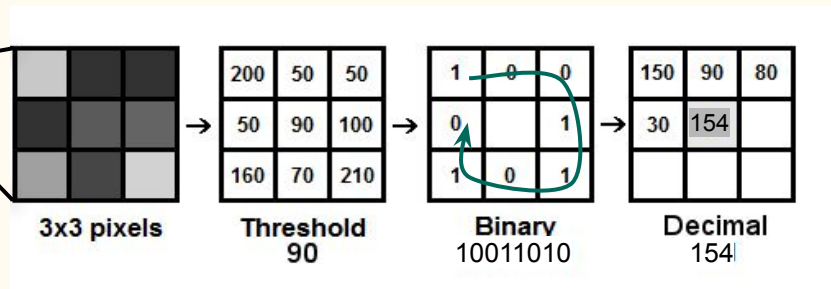
Paradigm shift of machine learning



Traditional method - Feature engineering

Local Binary Patterns Histograms algorithm (LBPH)

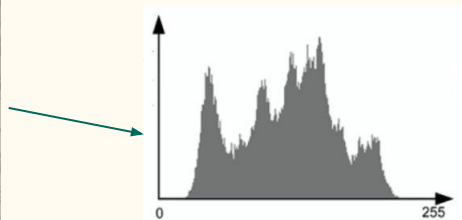
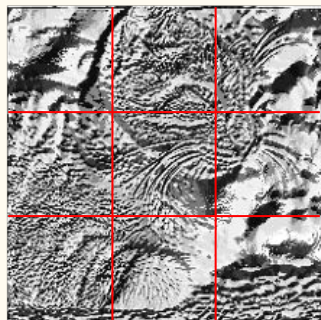
LBP: A texture operator which labels the pixels of an image by thresholding the neighborhood of each pixel and considers the result as a binary number.



Traditional method - Feature engineering

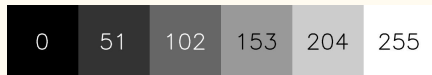
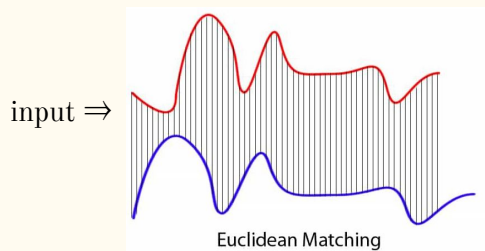
Local Binary Patterns Histograms algorithm (LBPH)

LBP: A texture operator which labels the pixels of an image by thresholding the neighborhood of each pixel and considers the result as a binary number.



Look like wave pattern of image

Algos: K-means clustering, SVM, ...



Feature engineering



IMAGENET Large Scale Visual Recognition Challenge 2010 (ILSVRC2010)

Held as a "taster competition" in conjunction with PASCAL Visual Object Classes Challenge 2010 (VOC2010)

Submissions ranked by flat cost

[Back to Main page](#)

team name	team members	filename	flat cost	hie cost	description
NEC-UIUC	NEC: Yuanqing Lin, Fengjun Lv, Shenghuo Zhu, Ming Yang, Timothee Cour, Kai Yu UIUC: LiangLiang Cao, Zhen Li, Min-Hsuan Tsai, Xi Zhou, Thomas Huang Rutgers: Tong Zhang	flat_opt.txt	0.28191	2.1144	using sift and lbp feature with two non-linear coding representations and stochastic SVM, optimized for top-5 hit rate

Feature engineering - limitations

Hard to handle **fine-grained object** that has complicated features



How to handle a complex task?

(in traditional method)

Use **stacked** feature extractor

LBP(edge)



SIFT(local invariants)



Gabor linear filter



input of estimators/models

Feature engineering vs Feature learning



Held in conjunction with [PASCAL Visual Object Classes Challenge 2012 \(VOC2012\)](#)

[Back to Main page](#)

All results

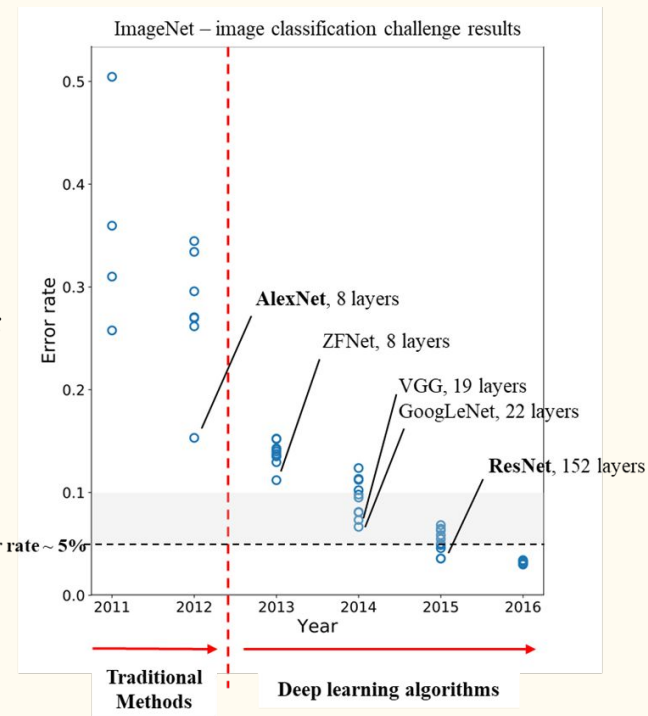
- [Task 1 \(classification\)](#)
- [Task 2 \(localization\)](#)
- [Task 3 \(fine-grained classification\)](#)
- [Team information and abstracts](#)

Task 1

Team name	Filename	Error (5 guesses)	Description
SuperVision	test-preds-141-146.2009-131-137-145-146.2011-145f.	0.15315	Using extra training data from ImageNet Fall 2011 release
SuperVision	test-preds-131-137-145-135-145f.txt	0.16422	Using only supplied training data
ISI	pred_FVs_wLACs_weighted.txt	0.26172	Weighted sum of scores from each classifier with SIFT+FV, LBP+FV, GIST+FV, and CSIFT+FV, respectively.

Feature learning

Feature engineering



Feature engineering vs Feature learning



- [Task 1 \(classification\)](#)
- [Task 2 \(localization\)](#)
- [Task 3 \(fine-grained classification\)](#)
- [Team information and abstracts](#)

Task 1

Team name	Filename	Error (5 guesses)	Description
SuperVision	test-preds-141-146.2009-131-137-145-146.2011-145f.	0.15315	Using extra training data from ImageNet Fall 2011
SuperVision	test-preds-131-137-145-135-145f.txt	0.16422	
ISI	pred_FVs_wLACs_weighted.txt	0.26172	

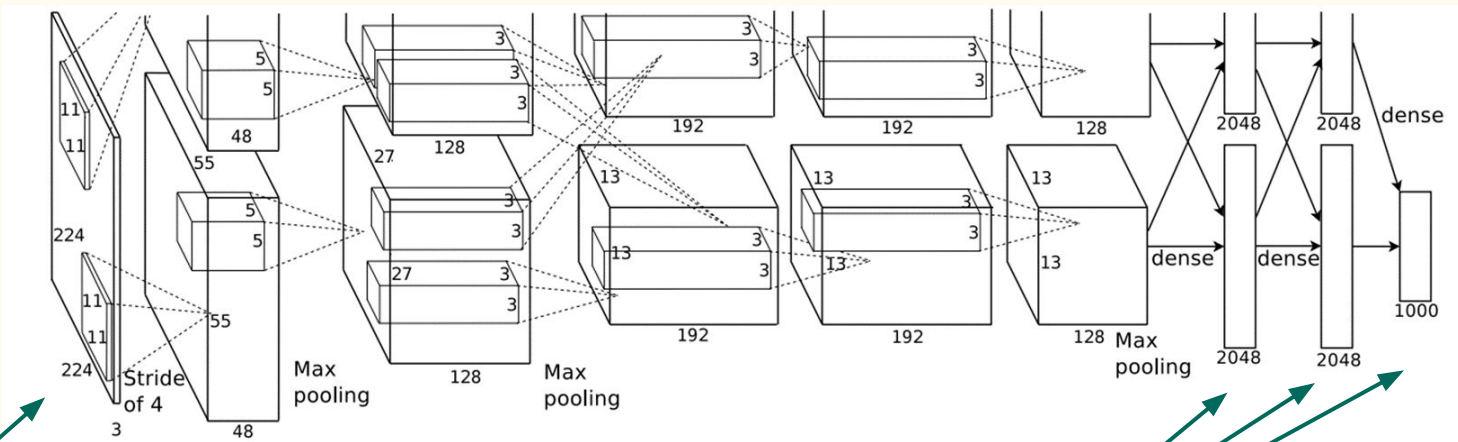
SuperVision

Alex Krizhevsky,
Ilya Sutskever,
Geoffrey Hinton

University of
Toronto

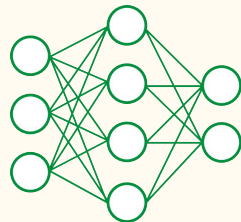
Our model is a large, deep convolutional neural network trained on raw RGB pixel values. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three globally-connected layers with a final 1000-way softmax. It was trained on two NVIDIA GPUs for about a week. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of convolutional nets. To reduce overfitting in the globally-connected layers we employed hidden-unit "dropout", a recently-developed regularization method that proved to be very effective.

AlexNet



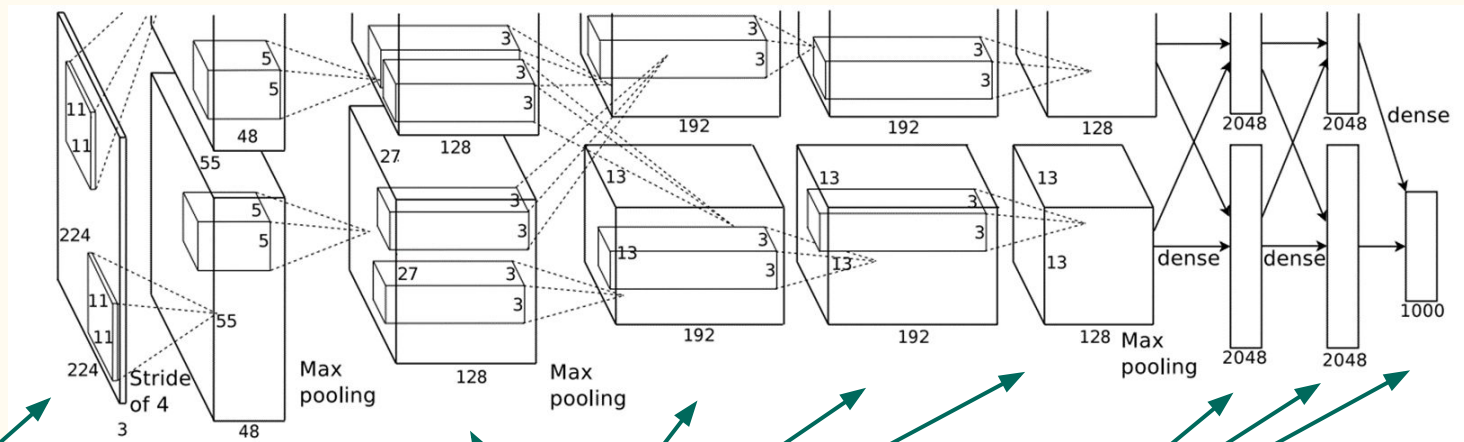
Input layer
(227x227x3)
RGB image pixel

Fully Connected layer 1, 2, 3 (output)
= ANN



1000 classes

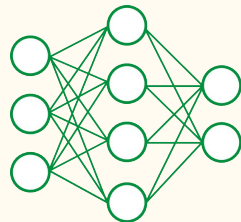
AlexNet



Input layer
(227x227x3)
RGB image pixel

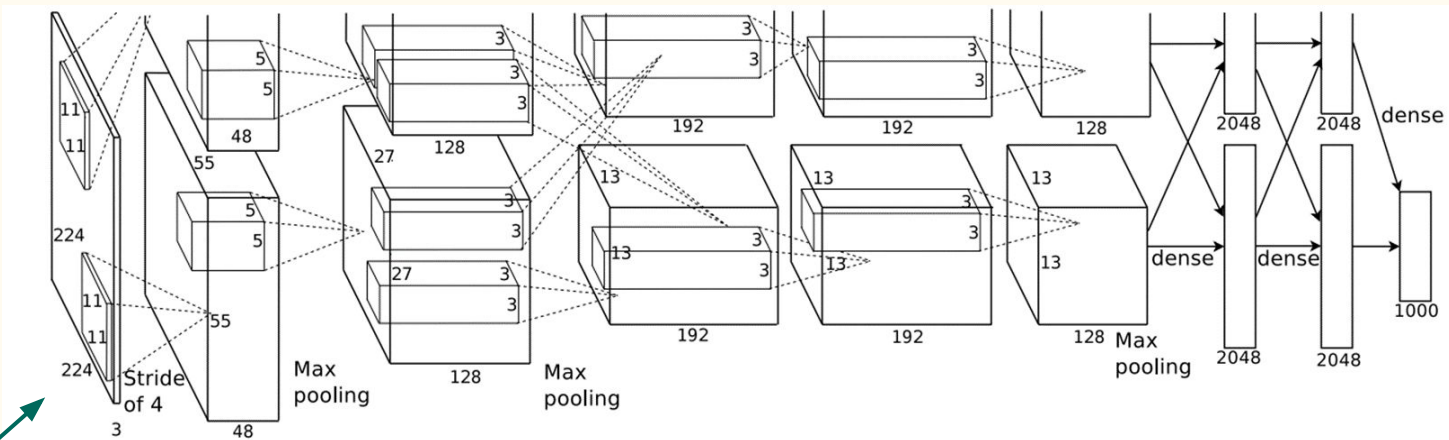
Convolution layer 1, 2, 3, 4, 5

Fully Connected layer 1, 2, 3(output)
= ANN



1000 classes

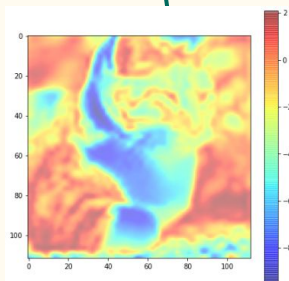
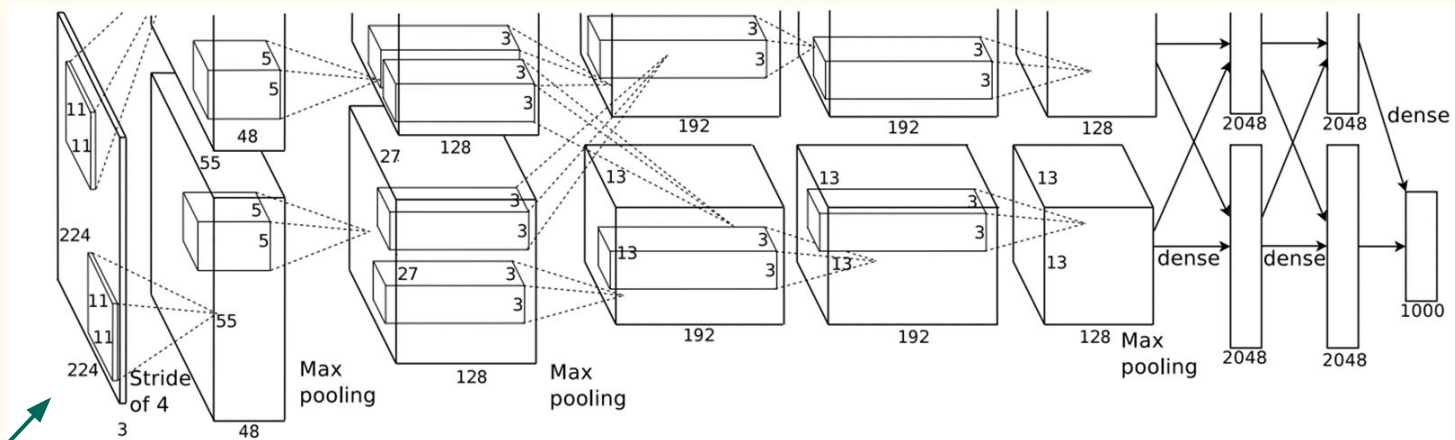
Idea of convolution layer



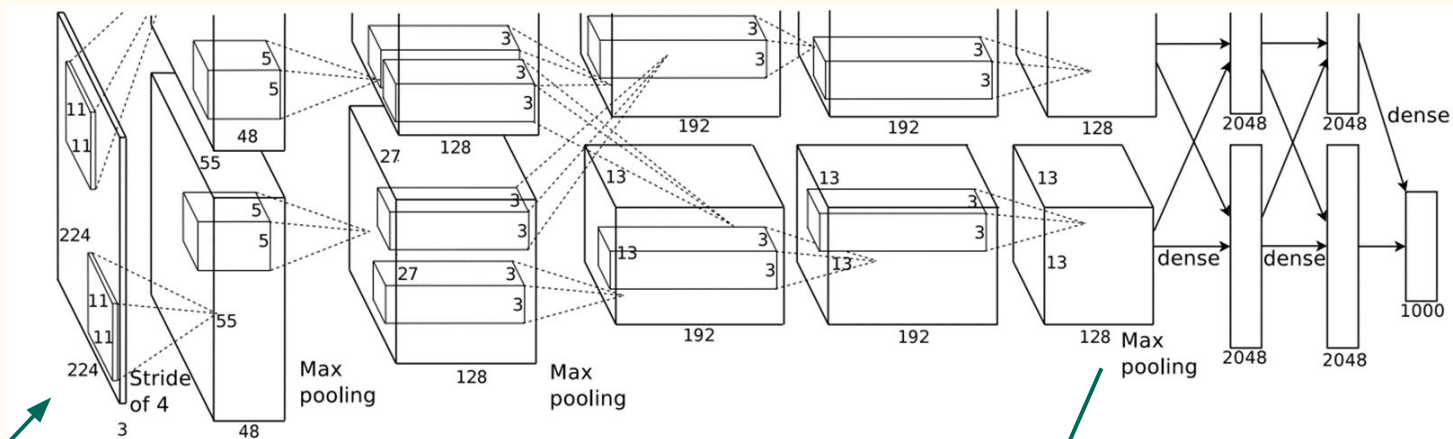
Input layer
(227x227x3)
RGB image pixel



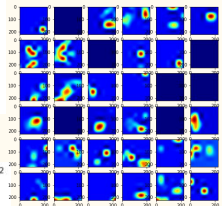
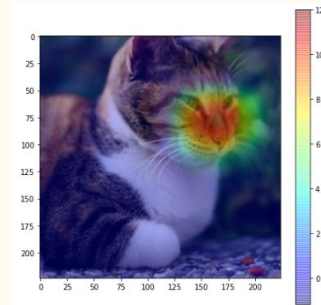
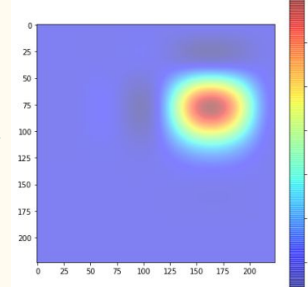
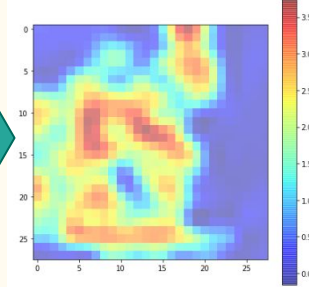
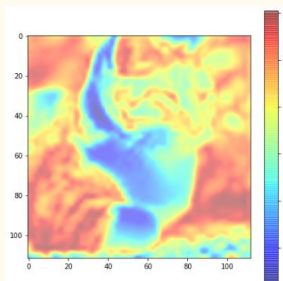
Idea of convolution layer



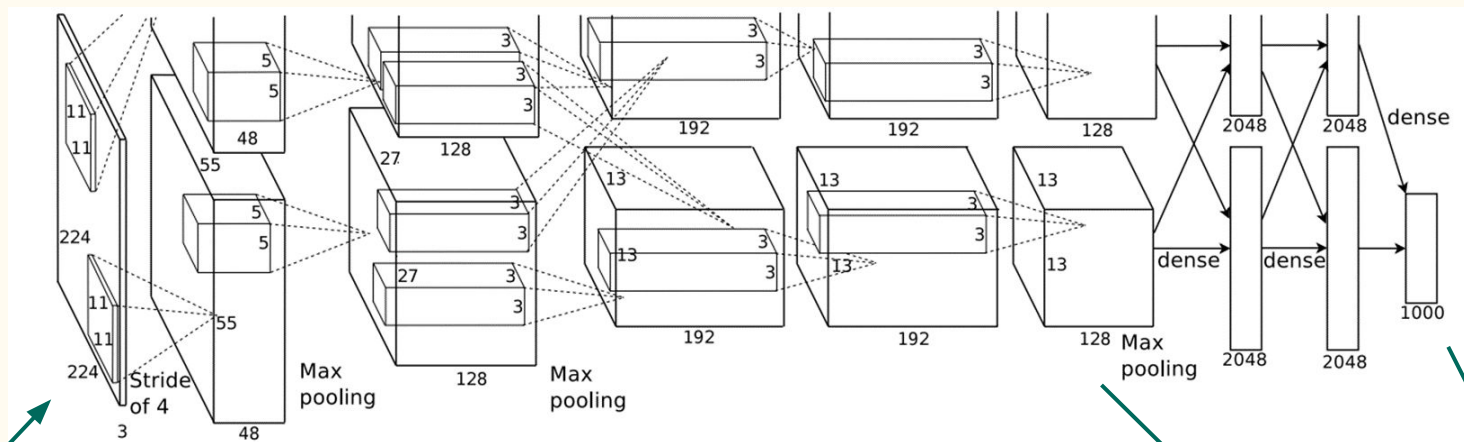
Idea of convolution layer



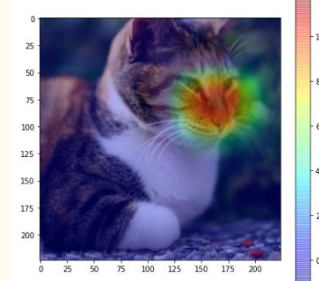
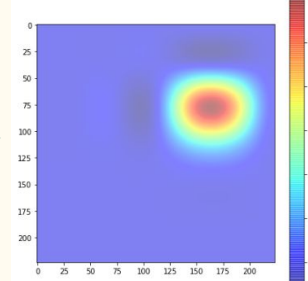
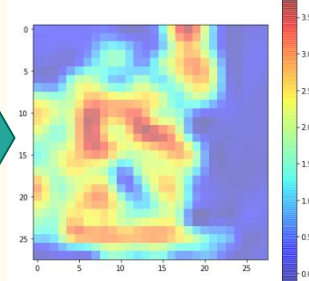
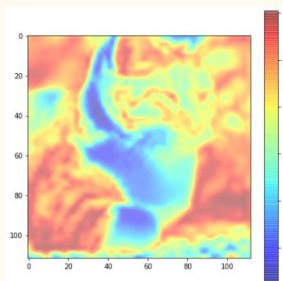
Feature map (generated by conv_layer)



Idea of convolution layer

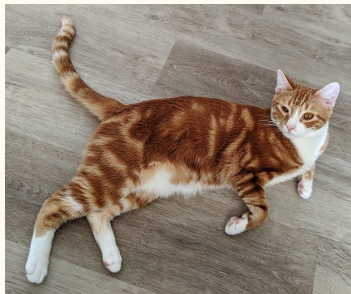
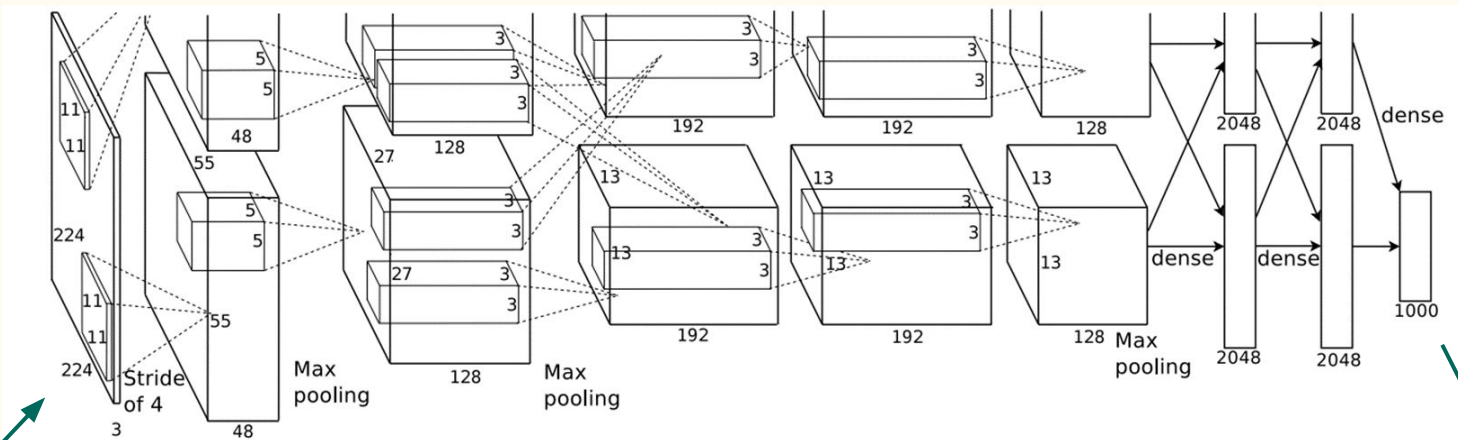


Feature map (generated by conv_layer)



Prediction
= class 281

Idea of convolution layer



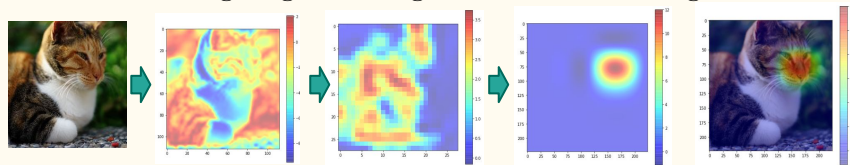
277	red fox, <i>Vulpes vulpes</i>
278	kit fox, <i>Vulpes macrotis</i>
279	Arctic fox, white fox, <i>Alopex lagopus</i>
280	grey fox, gray fox, <i>Urocyon cinereoargenteus</i>
281	<u>tabby, tabby cat</u>
282	tiger cat
283	Persian cat
284	Siamese cat, Siamese
285	Egyptian cat

Prediction
= class 281

Feature learning

- ❖ Feature extractor (Conv_layer) is learned by computer itself
- ❖ Uses multiple layers to progressively extract higher-level features from raw input (e.g. image)
- ❖ Higher-level features and concepts are defined in terms of lower-level ones
- ❖ “Hierarchical” Feature Learning

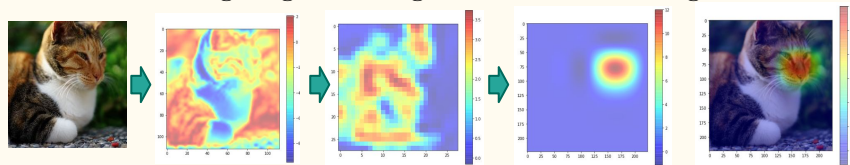
Lower-level features (e.g. edges) $\Rightarrow$ Higher-level features (e.g. cat nose/ whiskers)



Feature learning

- ❖ Feature extractor (Conv_layer) is learned by computer itself
- ❖ Uses multiple layers to progressively extract higher-level features from raw input (e.g. image)
- ❖ Higher-level features and concepts are defined in terms of lower-level ones
- ❖ “Hierarchical” Feature Learning
 - **Deep Learning**

Lower-level features (e.g. edges) $\Rightarrow$ Higher-level features (e.g. cat nose/ whiskers)



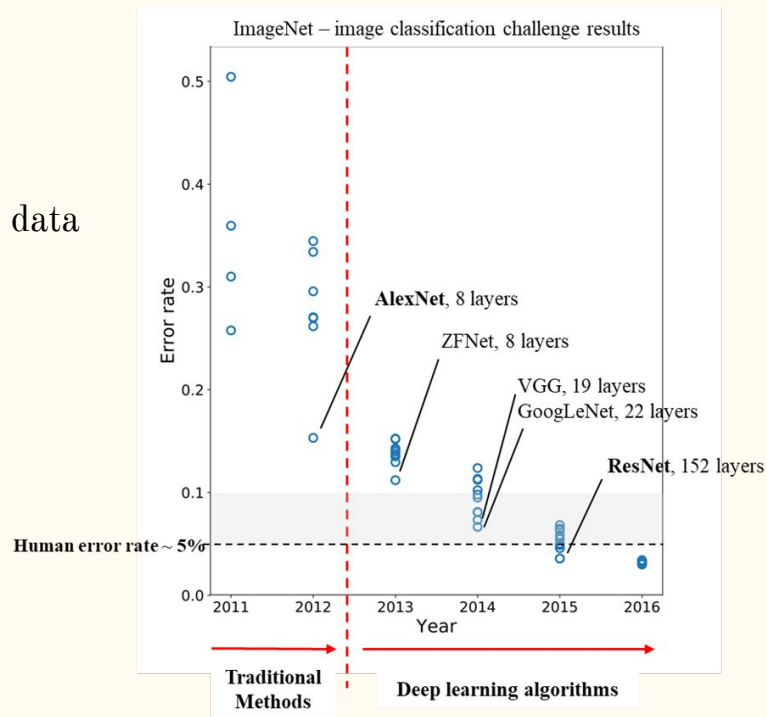
Paradigm shift of machine learning

❖ Big Data

- Volume of data & storage & flow
- Cannot use traditional way to store and analysis the data
- e.g. Large-scale dataset: ImageNet

❖ Deep Learning

- “Hierarchical” Feature Learning
 - Convolutional Neural Network



Paradigm shift of machine learning

❖ Big Data

- Volume of data & storage & flow
- Cannot use traditional way to store and analysis the data
- e.g. Large-scale dataset: ImageNet



❖ Deep Learning

- “Hierarchical” Feature Learning
 - Convolutional Neural Network

AlexNet

SuperVision	<u>Alex Krizhevsky,</u> Ilya Sutskever, Geoffrey Hinton University of Toronto	Our model is a large, <u>deep convolutional neural network</u> trained on raw RGB pixel values. The neural network, which has 60 million parameters and <u>650,000 neurons</u> , consists of five convolutional layers, some of which are followed by max-pooling layers, and <u>three globally-connected layers</u> with a final <u>1000-way softmax</u> . It was trained on <u>two NVIDIA GPUs for about a week</u> . To make training faster, we used non-saturating neurons and a very efficient GPU implementation of convolutional nets. To reduce overfitting in the globally-connected layers we employed hidden-unit "dropout", a recently-developed regularization method that proved to be very effective.
-------------	---	---

Paradigm shift of machine learning

❖ Big Data

- Volume of data & storage & flow
- Cannot use traditional way to store and analysis the data
- e.g. Large-scale dataset: ImageNet



❖ Deep Learning

- “Hierarchical” Feature Learning
 - Convolutional Neural Network

❖ Hardware and software

- GPU Graphics Processing Unit

AlexNet

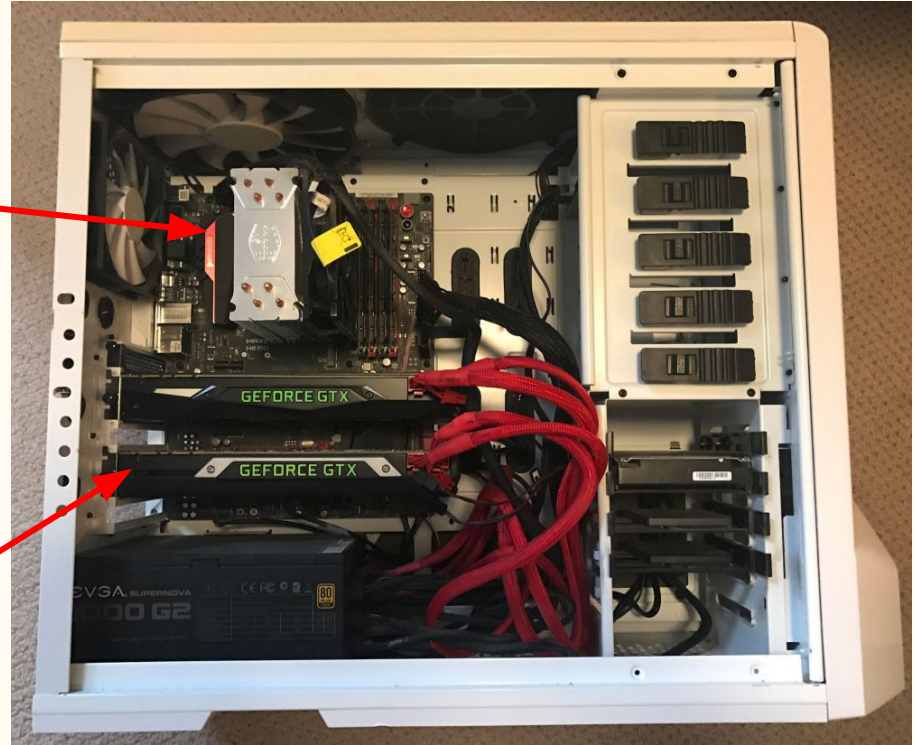
SuperVision	<u>Alex Krizhevsky,</u> Ilya Sutskever, Geoffrey Hinton University of Toronto	Our model is a large, <u>deep convolutional neural network</u> trained on raw RGB pixel values. The neural network, which has 60 million parameters and <u>650,000 neurons</u> , consists of five convolutional layers, some of which are followed by max-pooling layers, and <u>three globally-connected layers</u> with a final <u>1000-way softmax</u> . It was trained on <u>two NVIDIA GPUs for about a week</u> . To make training faster, we used non-saturating neurons and a very efficient GPU implementation of convolutional nets. To reduce overfitting in the globally-connected layers we employed hidden-unit "dropout", a recently-developed regularization method that proved to be very effective.
-------------	---	---

Inside the computer

CPU



GPU



CPU vs GPU



Prefix	Abbreviation	Order of magnitude (as a factor of 10)	Computer performance
giga-	G	10^9	gigaFLOPS (GFLOPS)
tera-	T	10^{12}	teraFLOPS (TFLOPS)

	Cores	Price	Performance	Remarks
CPU (Intel Core i7 -7700k)	4	\$2500	~442 GFLOPs	Fewer cores but each core is much faster; great at sequential tasks (e.g. scalar multiplication)
GPU (NVIDIA RTX 2080 Ti)	4352	\$10000	~13450 GFLOPs /~13.45TFLOPs	More cores but each core is much slower; great at parallel tasks (e.g. matrix multiplication)

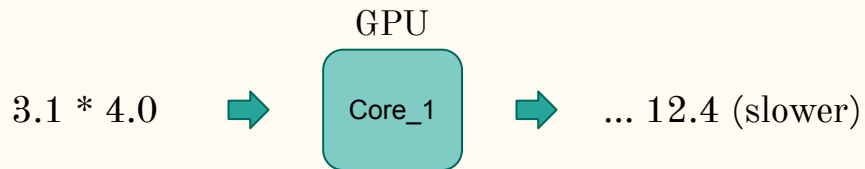
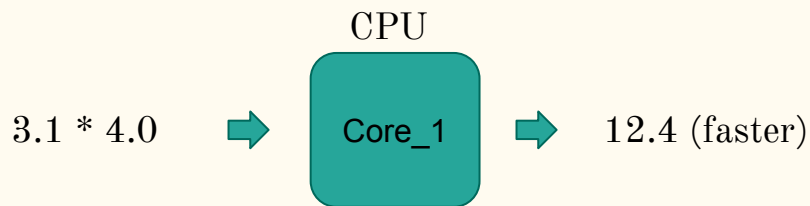


CPU vs GPU

❖ Scalar multiplication

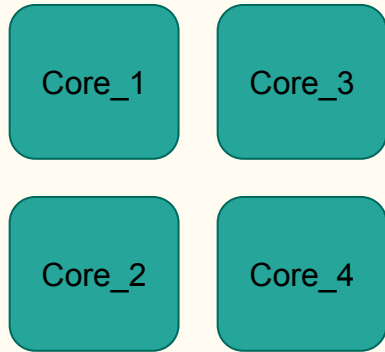
➤ e.g. $3.1 * 4.0 * 6$

➤ e.g. $3 * \mathbf{a}(\mathbf{x}, \mathbf{y}) = (3x, 3y)$

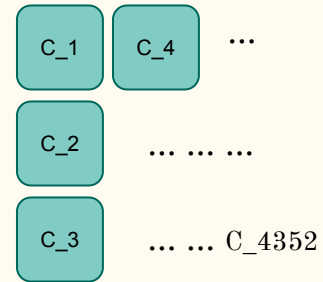


CPU vs GPU

CPU



GPU



CPU vs GPU

❖ Matrix multiplication

➤ e.g. $A \cdot B = C$

2x3

$$A = \begin{pmatrix} 11 & 22 & 33 \\ 44 & 55 & 66 \end{pmatrix}$$

3x2

$$B = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

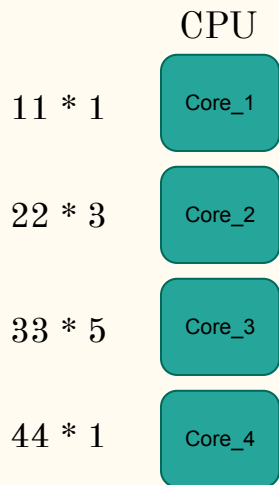
$$C = \begin{pmatrix} 11 \cdot 1 + 22 \cdot 3 + 33 \cdot 5 & 11 \cdot 2 + 22 \cdot 4 + 33 \cdot 6 \\ 44 \cdot 1 + 55 \cdot 3 + 66 \cdot 5 & 44 \cdot 2 + 55 \cdot 4 + 66 \cdot 6 \end{pmatrix}$$

2x2

CPU vs GPU

❖ Matrix multiplication

➤ e.g. $A \cdot B = C$



$${}^{2 \times 3} A = \begin{pmatrix} 11 & 22 & 33 \\ 44 & 55 & 66 \end{pmatrix}$$

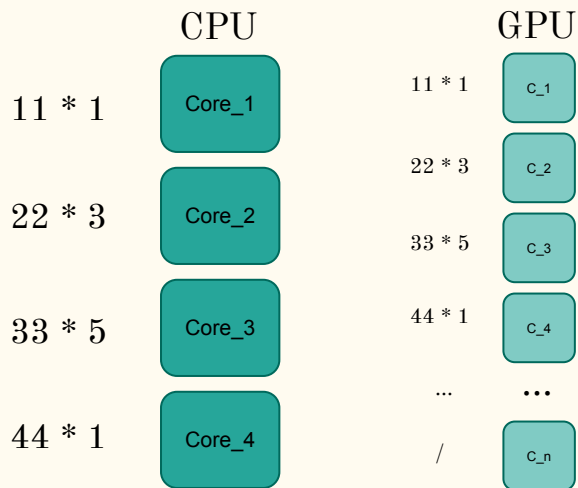
$${}^{3 \times 2} B = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

$${}^{2 \times 2} C = \begin{pmatrix} 11 \cdot 1 + 22 \cdot 3 + 33 \cdot 5 & 11 \cdot 2 + 22 \cdot 4 + 33 \cdot 6 \\ 44 \cdot 1 + 55 \cdot 3 + 66 \cdot 5 & 44 \cdot 2 + 55 \cdot 4 + 66 \cdot 6 \end{pmatrix}$$

CPU vs GPU

❖ Matrix multiplication

➤ e.g. $A \cdot B = C$



$${}^{2 \times 3} A = \begin{pmatrix} \boxed{11} & \boxed{22} & \boxed{33} \\ 44 & 55 & 66 \end{pmatrix}$$

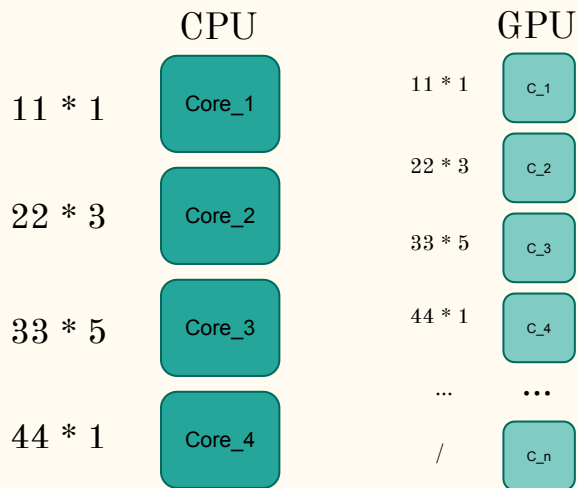
$${}^{3 \times 2} B = \begin{pmatrix} \boxed{1} & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

$${}^{2 \times 2} C = \begin{pmatrix} \boxed{11 \cdot 1 + 22 \cdot 3 + 33 \cdot 5} & 11 \cdot 2 + 22 \cdot 4 + 33 \cdot 6 \\ 44 \cdot 1 + 55 \cdot 3 + 66 \cdot 5 & 44 \cdot 2 + 55 \cdot 4 + 66 \cdot 6 \end{pmatrix}$$

CPU vs GPU

❖ Matrix multiplication

➤ e.g. $A \cdot B = C$



Done Calculation

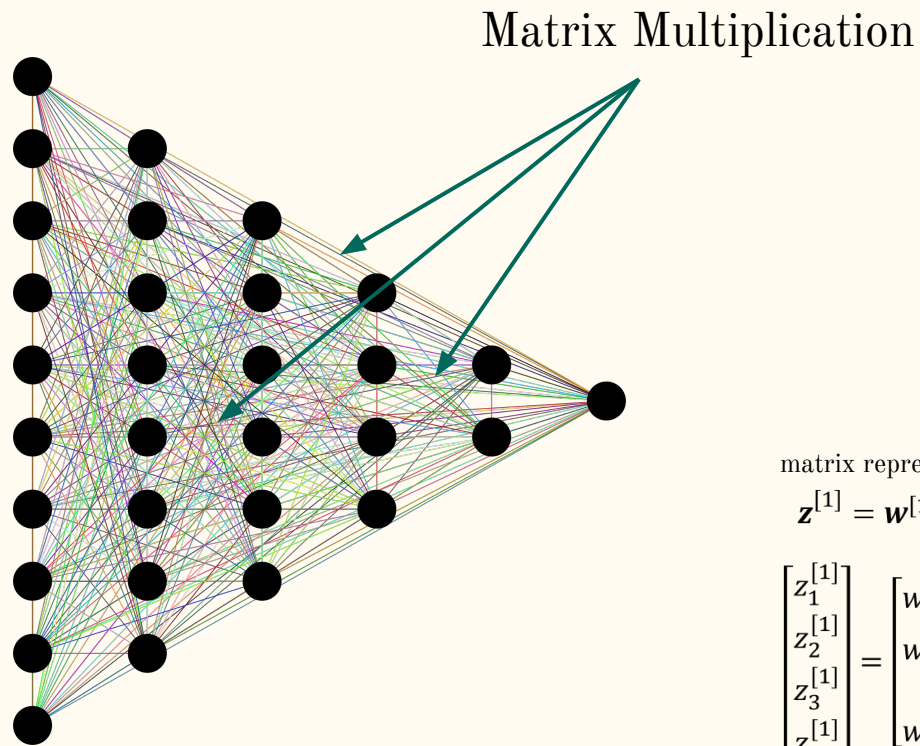
$${}^{2 \times 3} A = \begin{pmatrix} 11 & 22 & 33 \\ 44 & 55 & 66 \end{pmatrix}$$

$${}^{3 \times 2} B = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

$${}^{2 \times 2} C = \begin{pmatrix} 11 \cdot 1 + 22 \cdot 3 + 33 \cdot 5 & 11 \cdot 2 + 22 \cdot 4 + 33 \cdot 6 \\ 44 \cdot 1 + 55 \cdot 3 + 66 \cdot 5 & 44 \cdot 2 + 55 \cdot 4 + 66 \cdot 6 \end{pmatrix}$$

Great at parallel computing!

CPU vs GPU



matrix representations of Forward Pass

$$\mathbf{z}^{[1]} = \mathbf{w}^{[1]} \mathbf{a}^{[0]} + \mathbf{b}^{[1]}$$

$$\begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix}_{4 \times 1} = \begin{bmatrix} w_{11}^{[1]} & w_{12}^{[1]} & \dots & w_{1n}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} & \dots & w_{2n}^{[1]} \\ \vdots & \vdots & \dots & \vdots \\ w_{41}^{[1]} & w_{42}^{[1]} & \dots & w_{4n}^{[1]} \end{bmatrix}_{4 \times n} \times \begin{bmatrix} a_1^{[0]} \\ a_2^{[0]} \\ \vdots \\ a_n^{[0]} \end{bmatrix}_{n \times 1} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix}_{4 \times 1}$$

CPU vs GPU



CPU vs GPU

```
import torch
import numpy as np
```

CPU

```
[ ] # scalar multiplication
%%timeit -n 100
x = torch.randn(1,1).to("cpu")
result = torch.matmul(x,x)
del x, result
```

The slowest run took 41.83 times longer than the fastest. This could mean that an interim
100 loops, best of 5: 5.92 μ s per loop

```
[ ] # matrix multiplication
%%timeit -n 1
x = torch.randn(10000,10000).to("cpu")
result = torch.matmul(x,x)
del x, result
```

1 loop, best of 5: 24.9 s per loop

GPU

```
[ ] device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
device
```

```
device(type='cuda')
```

```
[ ] # scalar multiplication
%%timeit -n 100
x = torch.randn(1,1).to(device)
result = torch.matmul(x,x)
del x, result
```

100 loops, best of 5: 29.6 μ s per loop

```
[ ] # matrix multiplication
%%timeit -n 1
x = torch.randn(10000,10000).to(device)
result = torch.matmul(x,x)
del x, result
```

1 loop, best of 5: 723 ms per loop

Paradigm shift of machine learning

❖ Big Data

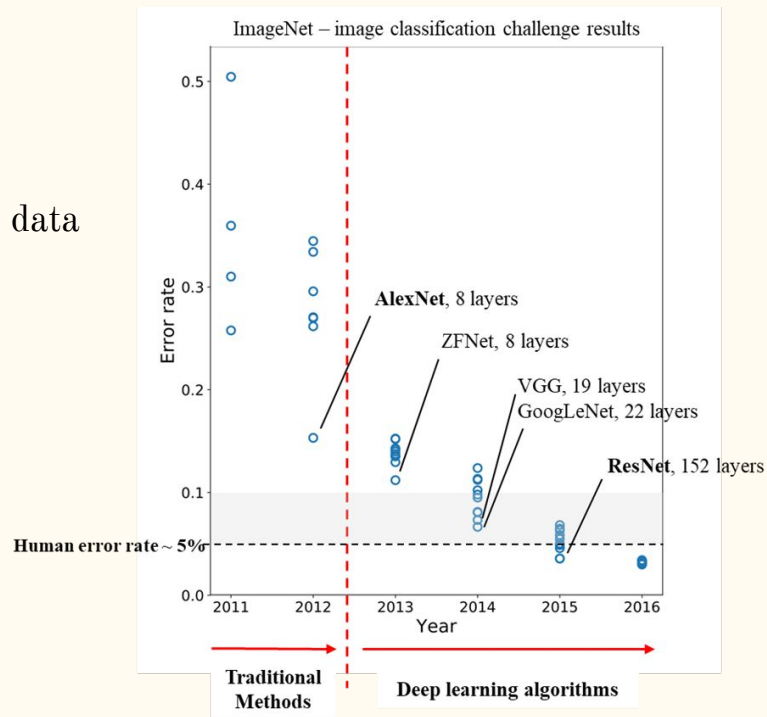
- Volume of data & storage & flow
- Cannot use traditional way to store and analysis the data
- e.g. Large-scale dataset: ImageNet

❖ Deep Learning

- “Hierarchical” Feature Learning
 - Convolutional Neural Network

❖ Hardware and software

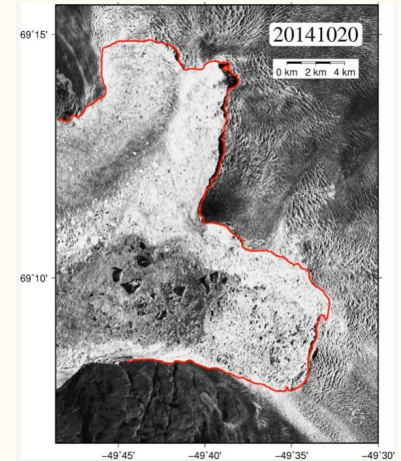
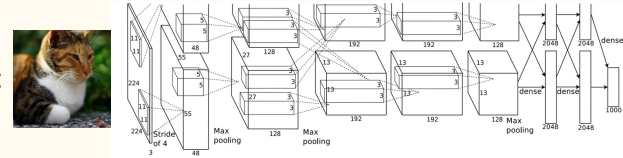
- GPU Graphics Processing Unit



Deep learning algorithms

- ❖ Convolutional Neural Network (CNN)
 - Computer vision: Image recognition; Remote sensing

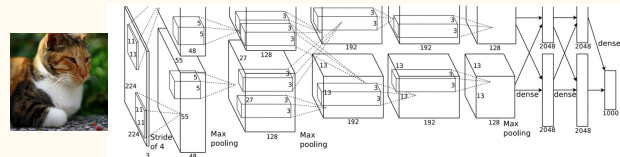
CNN:



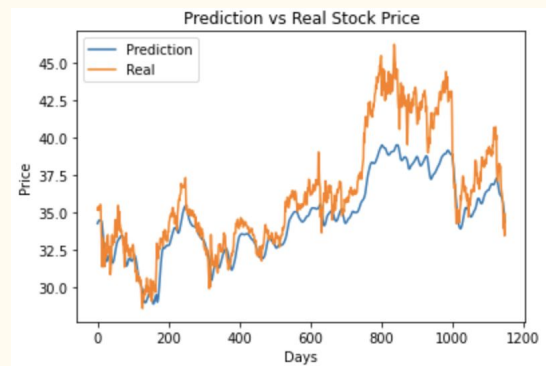
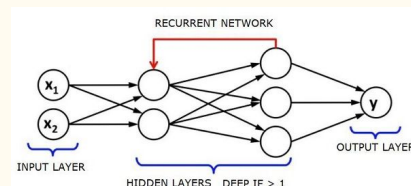
Deep learning algorithms

- ❖ Convolutional Neural Network (CNN)
 - Computer vision: Image recognition; Remote sensing
- ❖ Recurrent Neural Network (RNN)
 - Long Short Term Memory Networks (LSTMs)
 - NLP, Speech recognition; time-series prediction

CNN:



RNN:

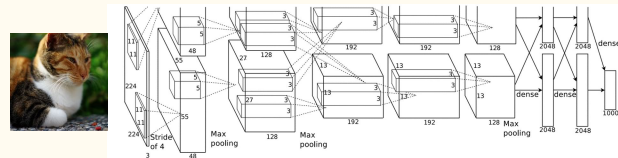


Deep learning algorithms

❖ Convolutional Neural Network (CNN)

- Computer vision: Image recognition; Remote sensing

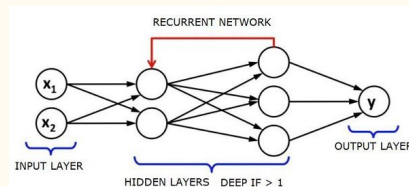
CNN:



❖ Recurrent Neural Network (RNN)

- Long Short Term Memory Networks (LSTMs)
- NLP, Speech recognition; time-series prediction

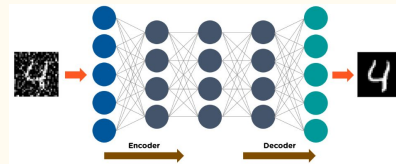
RNN:



❖ Generative Adversarial Networks (GANs)

- Generate realistic data

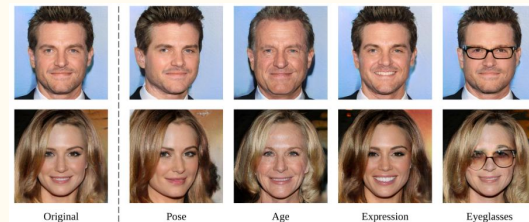
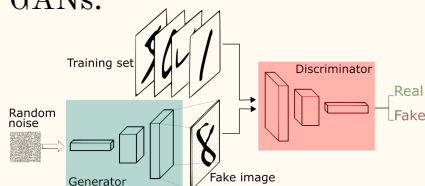
AutoEncoders:



❖ AutoEncoders

- Reconstruct the original image

GANs:



...

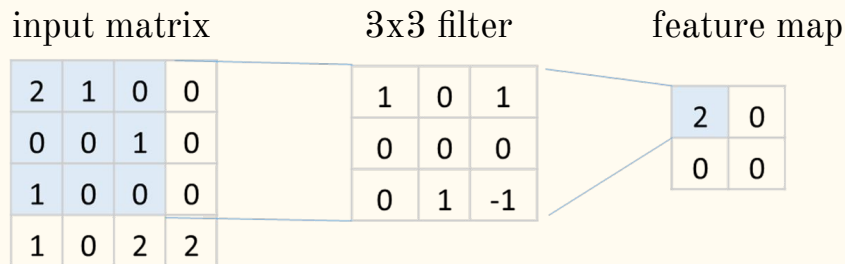
Convolutional Neural Networks

Convolution operation

Convolve the **filter** with the input matrix

i.e. “slide over the input matrix spatially, computing dot products”

➤ Filter the information and extract features from input



3 ₀	3 ₁	2 ₂	1	0
0 ₂	0 ₂	1 ₀	3	1
3 ₀	1 ₁	2 ₂	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

Edge detection in feature engineering

input image

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

3x3 filter

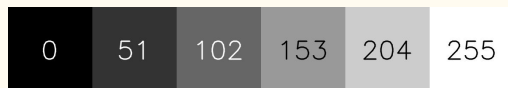
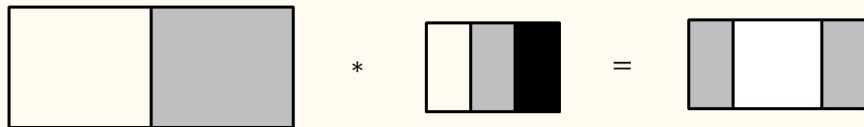
1	0	-1
1	0	-1
1	0	-1

feature map

0	30	30	0
0	30	30	0
0	30	30	0

*

=



Edge detection in feature learning

In feature learning, weight is the parameter that calculated by computer itself (through the process of backpropagation)

input image

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

3x3 filter

Weights

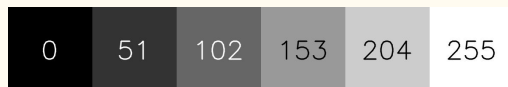
feature map

0	30	30	0
0	30	30	0
0	30	30	0



* =

Visual representation of the convolution operation using grayscale blocks.



Padding

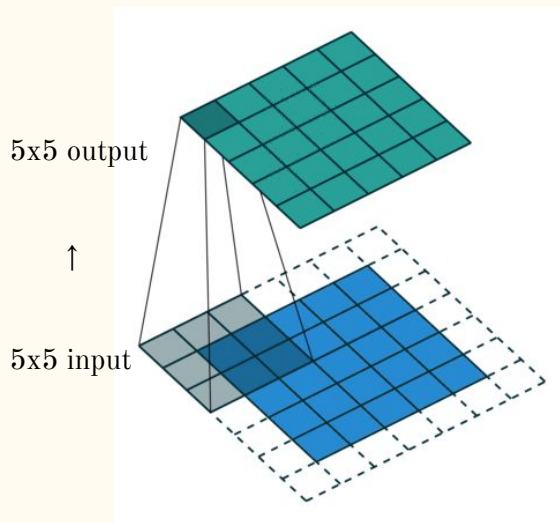
Pad the edges with extra, “fake” pixels (usually of value 0, hence the term “zero padding”)

- Preserve input spatial dimensions in output activations
- Improve performance by keeping information at the borders

5x5 input image with zero padding

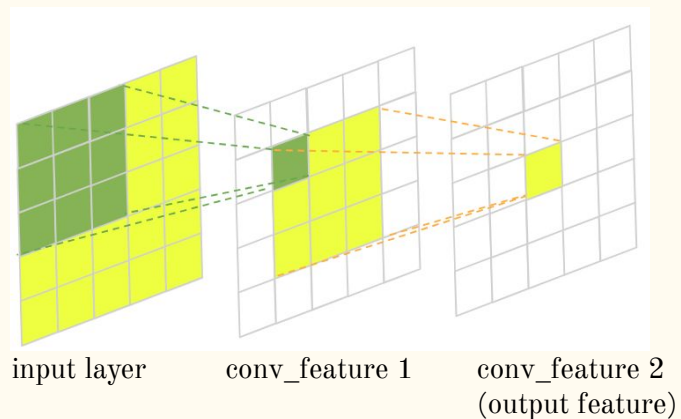
0	0	0	0	0	0	0
0	2	3	0	-2	-3	0
0	-1	2	-2	-1	4	0
0	2	-3	0	0	0	0
0	0	2	1	2	5	0
0	0	-2	3	1	0	0
0	0	0	0	0	0	0

Padding Type	Description	Impact
Valid	No padding	Dimensions reduce
Same	Zeros around the edges	Dimensions stay the same



Receptive field

Receptive field is the region in the input space that a particular convoluted feature is affected by.

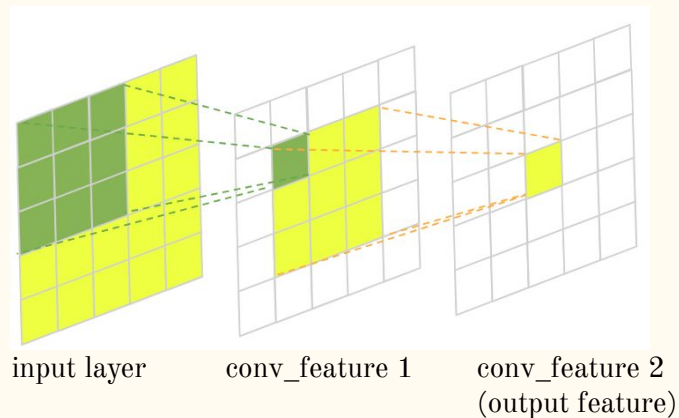


In this example,

the receptive field size of output feature (1x1 value) is the
5x5 input layer

Receptive field

Receptive field is the region in the input space that a particular convoluted feature is affected by.



Problem:

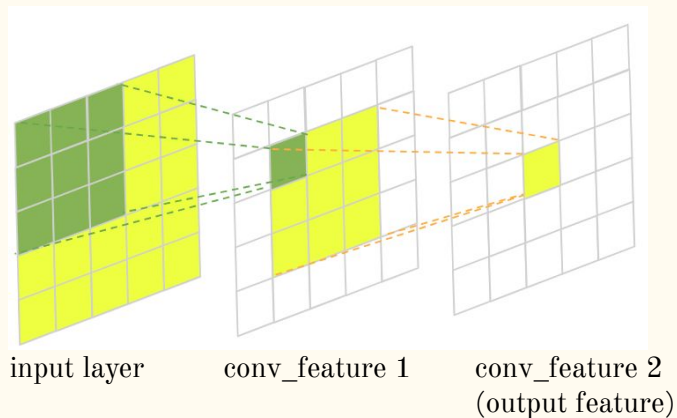
For large images, we need many layers for each output to “see” the whole image

In this example,

the receptive field size of output feature (1x1 value) is the 5x5 input layer

Receptive field

Receptive field is the region in the input space that a particular convoluted feature is affected by.



In this example,
the receptive field size of output feature (1x1 value) is the
5x5 input layer

Problem:

For large images, we need many layers for each
output to “see” the whole image

Solutions:

1. Downsample inside the layer (Stride)
2. Downsample outside the layer (Pooling)

Stride

Stride controls how the filter convolves around the input layer.

e.g. With a stride of 2, you advance the filter by two pixels at each step.

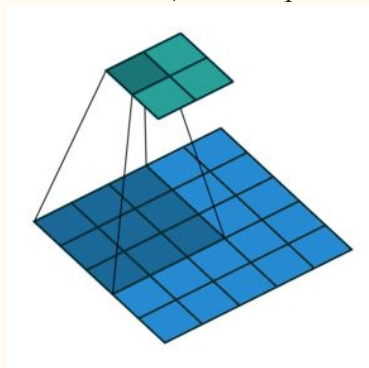
- Reduce the spatial dimension of output layer while keeping the same receptive field size
- Improve the efficiency of convolution operation

Stride 1 (by default) | 3x3 output

3 ₀	3 ₁	2 ₂	1	0
0 ₂	0 ₂	1 ₀	3	1
3 ₀	1 ₁	2 ₂	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

Stride 2 | 2x2 output

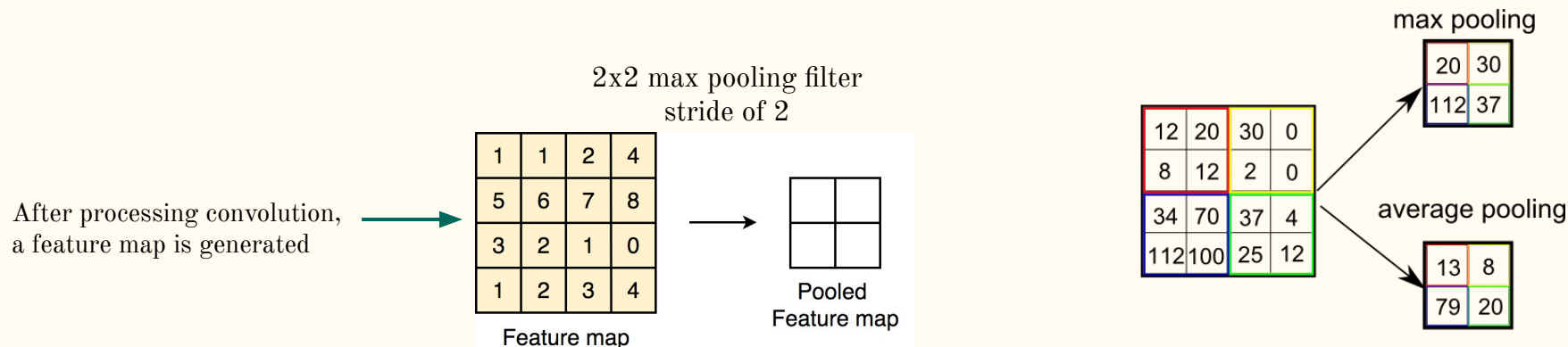


Pooling layer

Downsampling feature maps by summarizing the presence of features in patches of the feature map.

We usually use max pooling that calculates the maximum value for patches of the feature map.

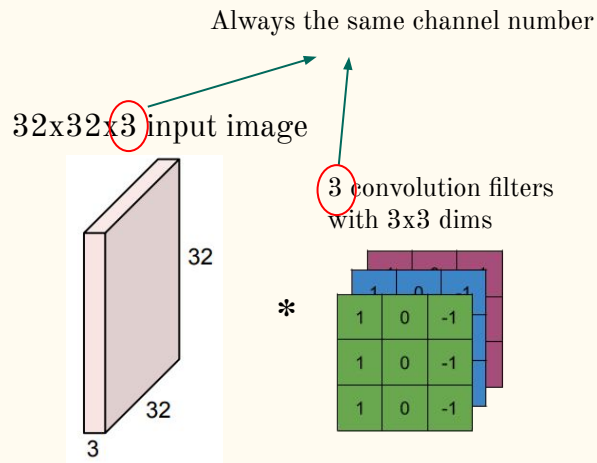
- Reduce the spatial dimension of feature map and make it more manageable
- Looking for more representative feature values



Convolution over volume

Most input images have 3 channels (RGB).

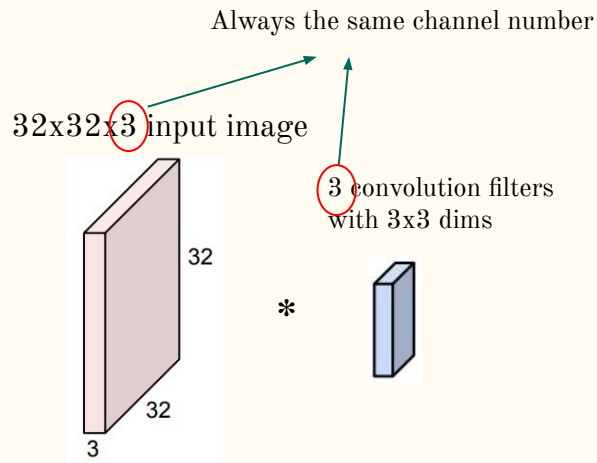
Convolve the filters with the image, i.e. “slide over the image spatially, computing dot products”



Convolution over volume

Most input images have 3 channels (RGB).

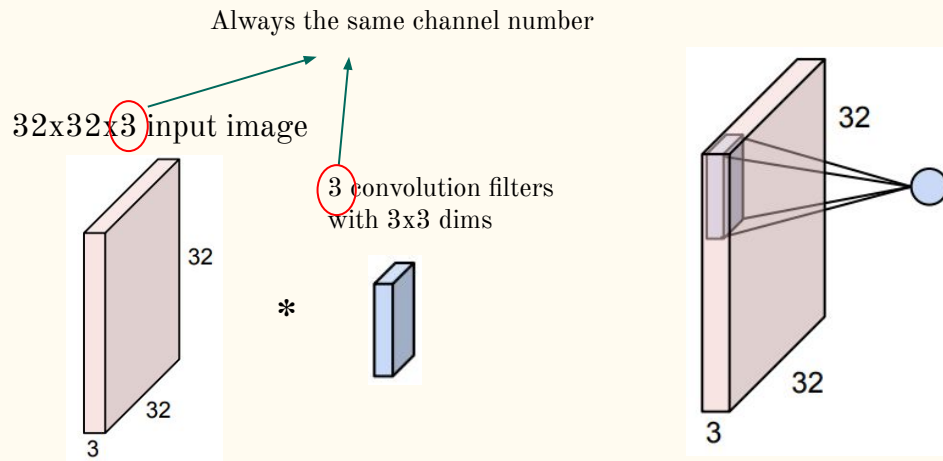
Convolve the filters with the image, i.e. “slide over the image spatially, computing dot products”



Convolution over volume

Most input images have 3 channels (RGB).

Convolve the filters with the image, i.e. “slide over the image spatially, computing dot products”



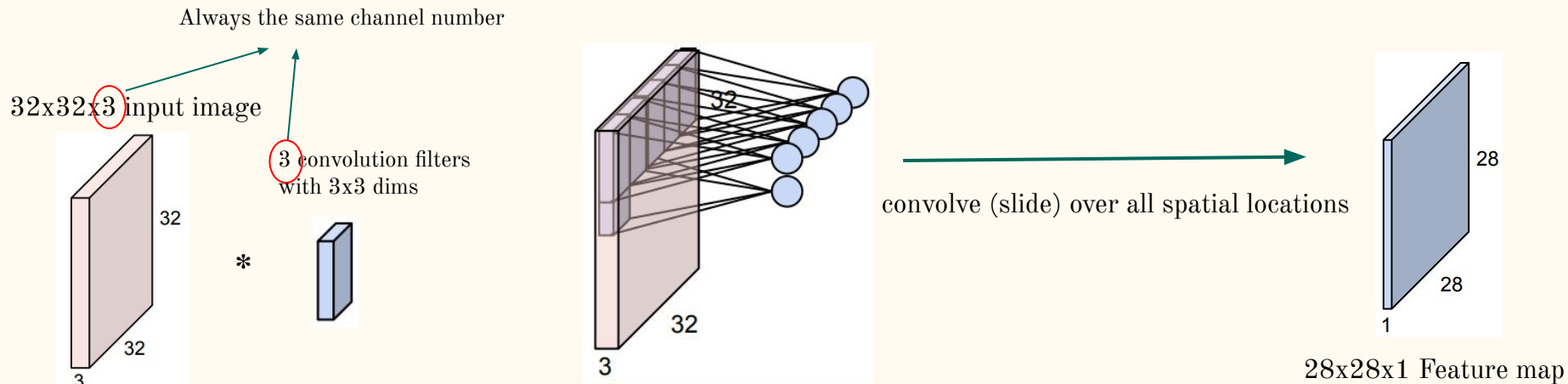
1 number:

the result of taking a dot product between the filter and a small 3x3x3 chunk of the image

Convolution over volume

Most input images have 3 channels (RGB).

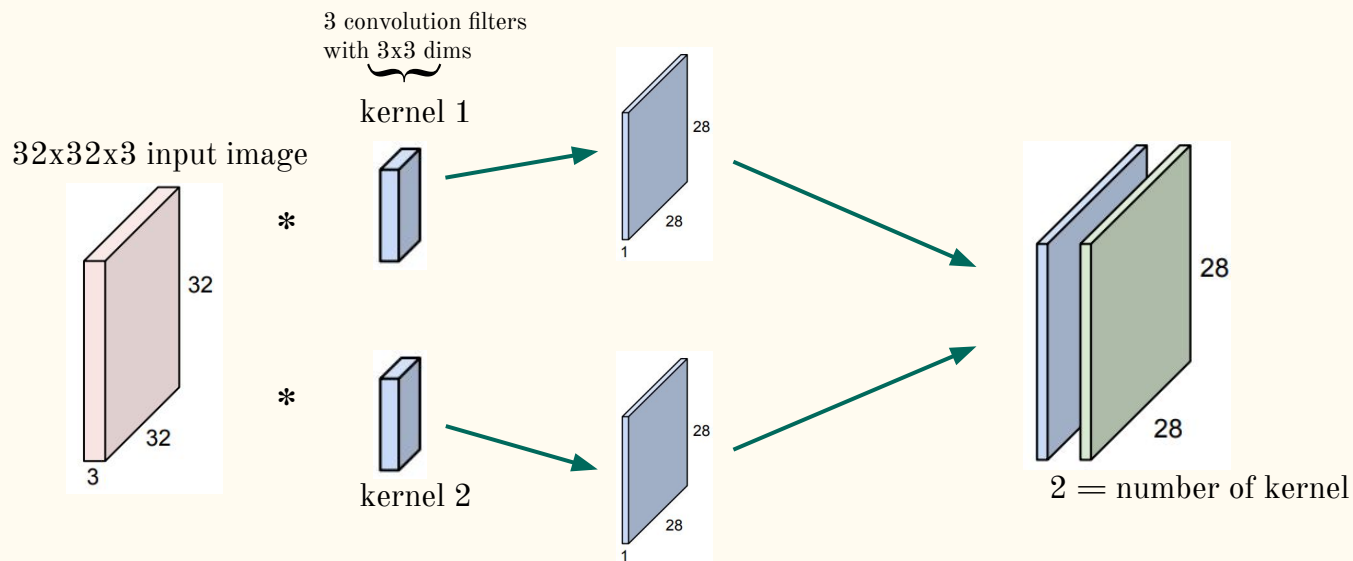
Convolve the filters with the image, i.e. “slide over the image spatially, computing dot products”



Convolution over volume

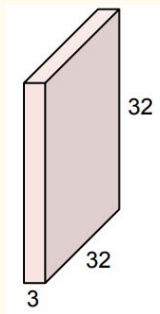
Most input images have 3 channels (RGB).

Convolve the filters with the image, i.e. “slide over the image spatially, computing dot products”



Convolution layers

32x32x3 input image



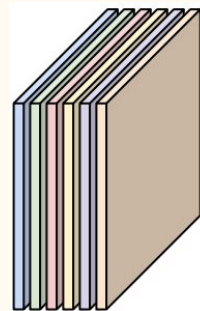
Convolution
layer



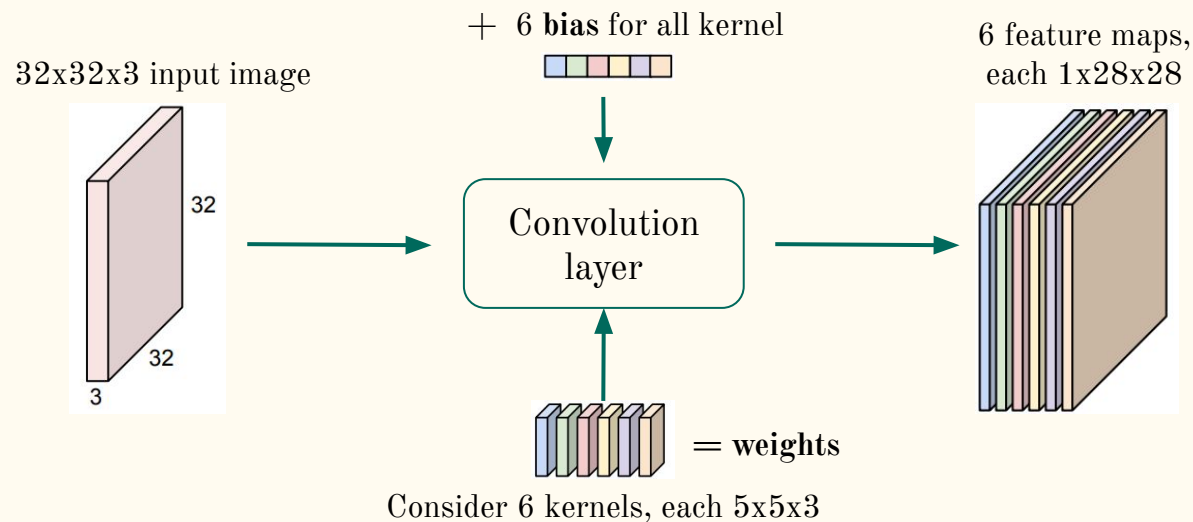
Consider 6 kernels, each 5x5x3



6 feature maps,
each 1x28x28

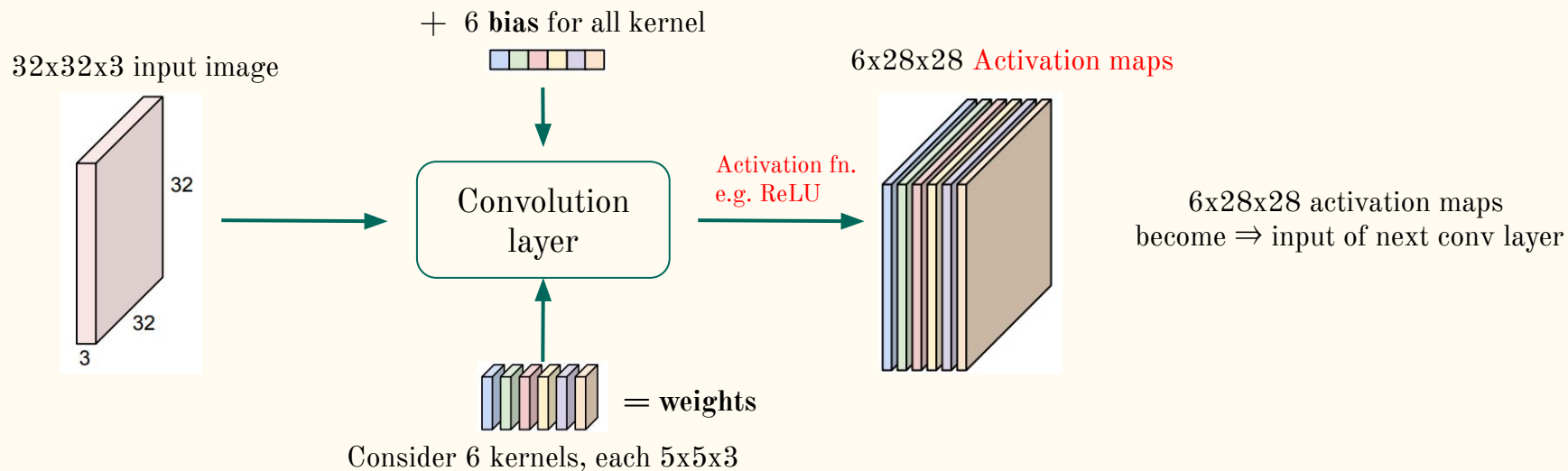


Convolution layers



$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = z$$
$$g(z) = a$$

Convolution layers

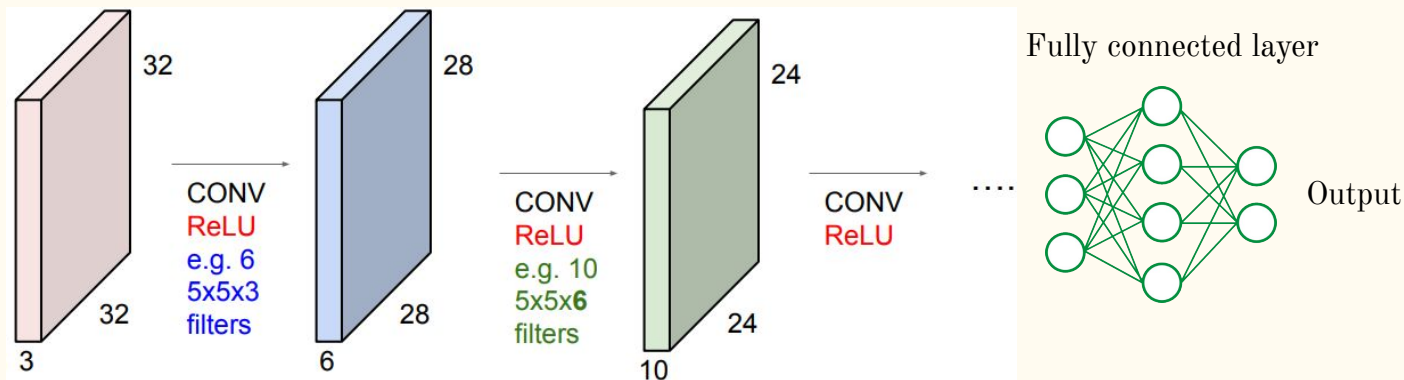


$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = z$$
$$g(z) = a$$

Convolutional Neural Network

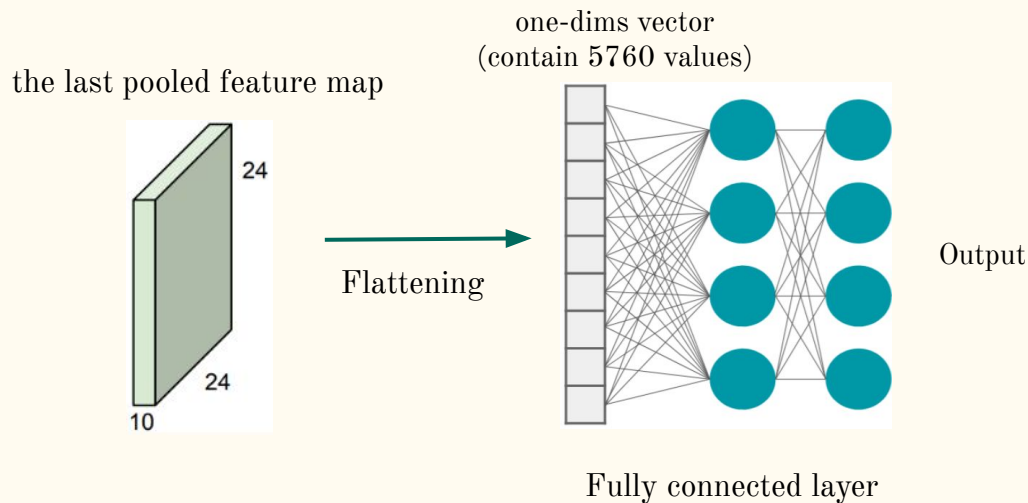
Convolutional Neural Network (CNN) is a sequence of convolution layers, interspersed with activation functions.

- **Key:** To filter the information and extract features from input data

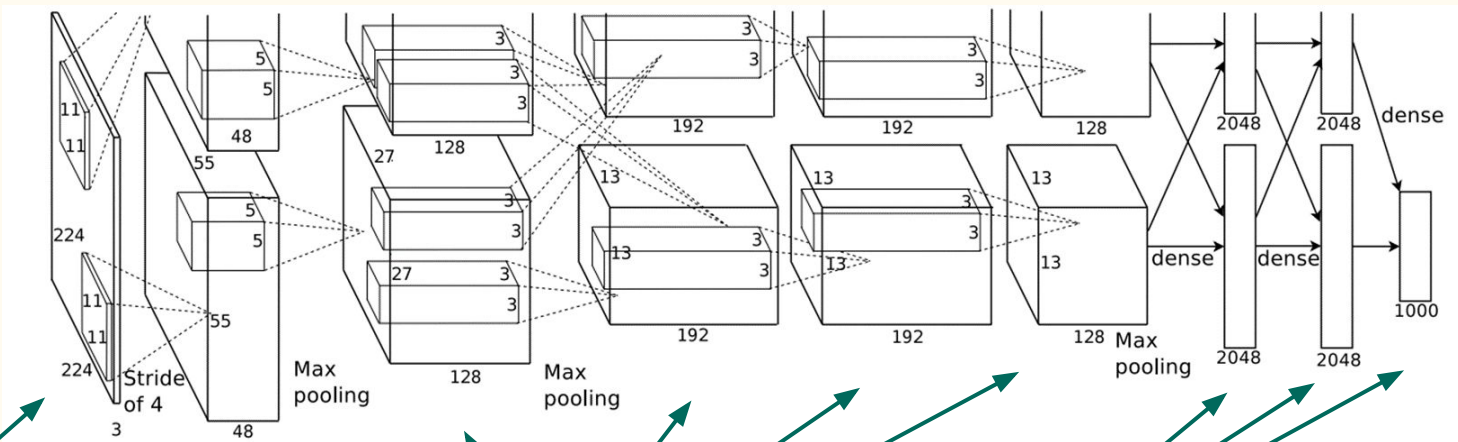


Fully connected layer

The input to the fully connected layer is the output from the final pooling or convolutional layer, which is “flattened” and unrolled all its values into a vector.



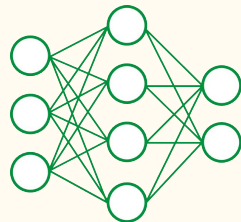
Case study: AlexNet



Input layer
(227x227x3)
RGB image pixel

Convolution layer 1, 2, 3, 4, 5

Fully Connected layer 1, 2, 3(output)
= ANN



1000 classes

Case study: AlexNet

Architecture:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

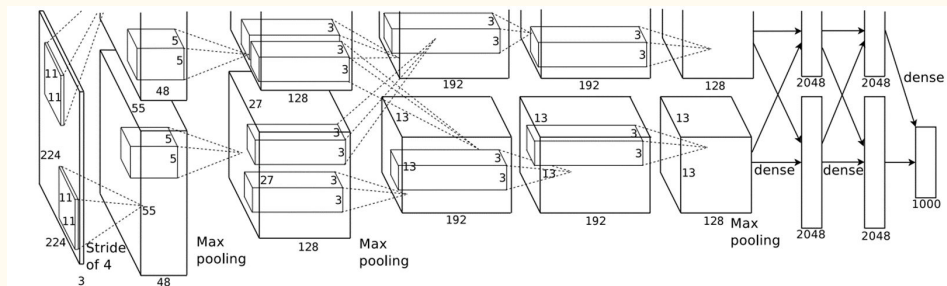
CONV5

Max POOL3

FC6

FC7

FC8

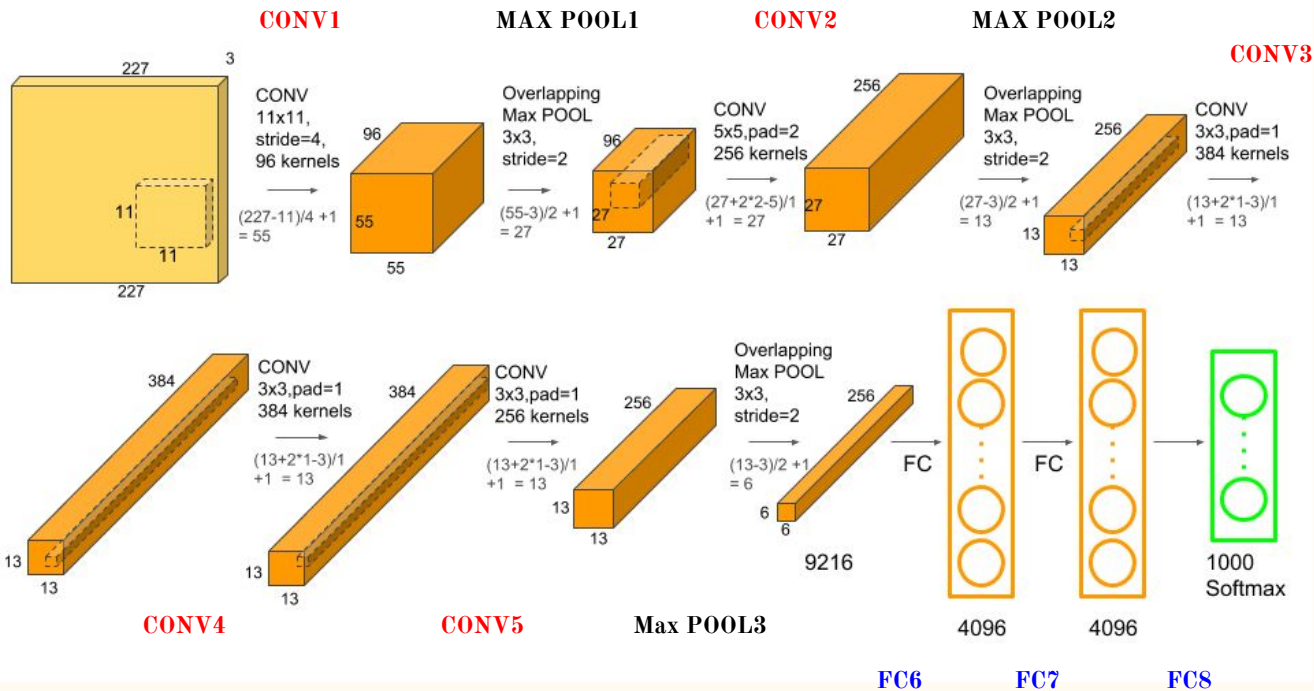
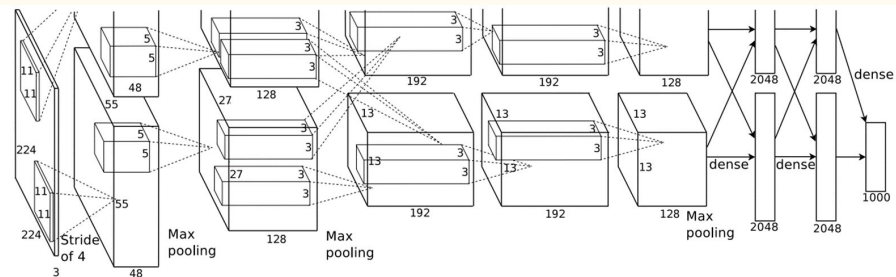


Trained on GTX 580 GPU with 3 GB of memory. Network spread across 2 GPUs.

One runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom.

The GPUs communicate only at certain layers.

Case study: AlexNet



Operation	Descriptions	Input Size	Output Size
CONV	11x11, stride=4, 96 kernels	227x227x3	56x56x96
MAXPOOL	3x3, stride=2	56x56x96	27x27x96
CONV	5x5, pad=2, 256 kernels	27x27x96	27x27x256
MAXPOOL	3x3, stride=2	27x27x256	13x13x256
CONV	3x3, pad=1, 384 kernels	13x13x256	13x13x384
CONV	3x3, pad=1, 384 kernels	13x13x384	13x13x384
CONV	3x3, pad=1, 256 kernels	13x13x384	13x13x256
MAXPOOL	3x3, stride=2	13x13x256	6x6x256
FC	ReLU	9216	4096
FC	ReLU	4096	4096
Output	Softmax	4096	1000

Calculation: output dimension

$$\text{floor}\left(\frac{W + 2 \times \text{pad} - ks}{S}\right) + 1$$

W: input size

pad: padding

ks: kernel size

S: stride

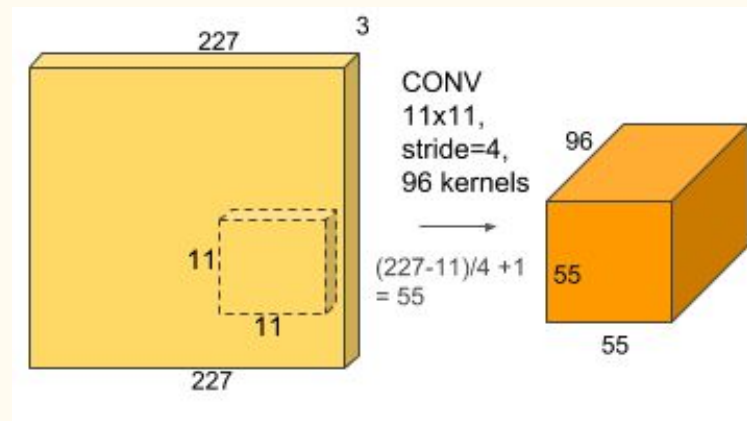
Example: CONV1

Input: 227x227x3 image

CONV1: 96 11x11x3 kernels, stride 4, valid padding

Output size: $(227-11)/4 + 1 = 55$

Output dims: 55x55x96



Calculation: parameters

Parameters = # Weight + # Bias

Input layer & pooling layer: parameters = 0

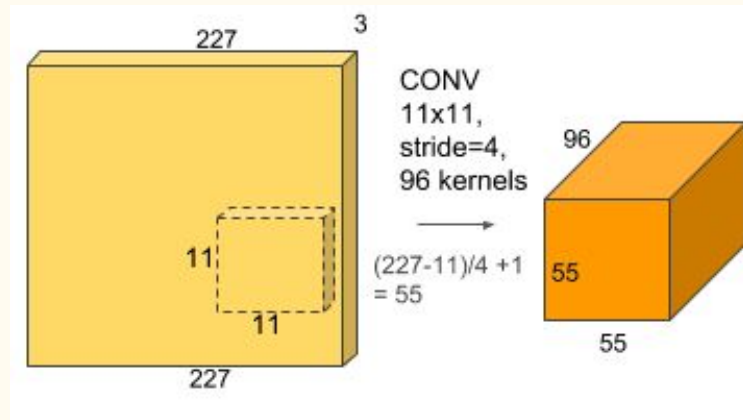
Example: CONV1

Input: 227x227x3 image

CONV1: 96 11x11x3 kernels, stride 4, valid padding

Each kernel has: $11 \times 11 \times 3 + 1 = 364$ parameters (+1 for bias)

$\Rightarrow 364 \times 96 = 34944$ parameters in convolution layer 1



Calculation: parameters

Parameters = # Weight + # Bias

Input layer & pooling layer: parameters = 0

Example: CONV1

Input: 227x227x3 image

CONV1: 96 11x11x3 kernels, stride 4, valid padding

Each kernel has: $11 \times 11 \times 3 + 1 = 364$ parameters (+1 for bias)

$\Rightarrow 364 \times 96 = 34944$ parameters in convolution layer 1

```
model.summary()
# params = # of weights + # of bias
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 55, 55, 96)	34944
batch_normalization (Batch Normalization)	(None, 55, 55, 96)	384
max_pooling2d (MaxPooling2D)	(None, 27, 27, 96)	0
conv2d_1 (Conv2D)	(None, 27, 27, 256)	614656
batch_normalization_1 (Batch Normalization)	(None, 27, 27, 256)	1024
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 256)	0
conv2d_2 (Conv2D)	(None, 13, 13, 384)	885120
batch_normalization_2 (Batch Normalization)	(None, 13, 13, 384)	1536
conv2d_3 (Conv2D)	(None, 13, 13, 384)	1327488
batch_normalization_3 (Batch Normalization)	(None, 13, 13, 384)	1536
conv2d_4 (Conv2D)	(None, 13, 13, 256)	884992
batch_normalization_4 (Batch Normalization)	(None, 13, 13, 256)	1024
max_pooling2d_2 (MaxPooling2D)	(None, 6, 6, 256)	0
flatten (Flatten)	(None, 9216)	0
dense (Dense)	(None, 4096)	37752832
dropout (Dropout)	(None, 4096)	0
dense_1 (Dense)	(None, 10)	40970

=====
Total params: 41,546,506
Trainable params: 41,543,754
Non-trainable params: 2,752

Case study: AlexNet

The first convolutional layer filters the $224 \times 224 \times 3$ input image with 96 kernels of size $11 \times 11 \times 3$ with a stride of 4 pixels (this is the distance between the receptive field centers of neighboring

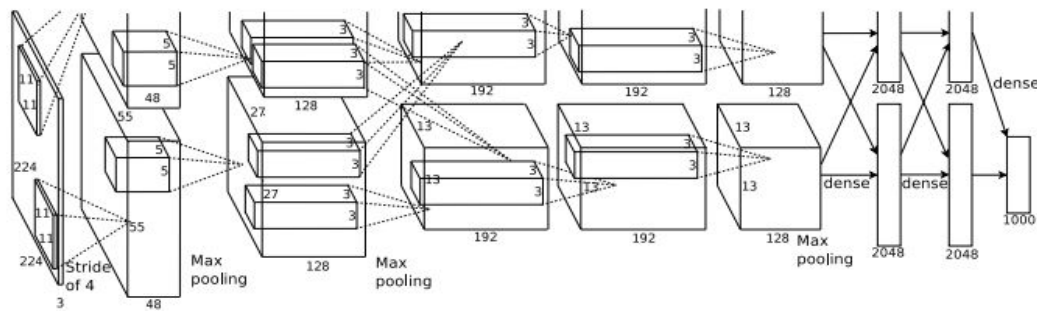


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

neurons in a kernel map). The second convolutional layer takes as input the (response-normalized and pooled) output of the first convolutional layer and filters it with 256 kernels of size $5 \times 5 \times 48$. The third, fourth, and fifth convolutional layers are connected to one another without any intervening pooling or normalization layers. The third convolutional layer has 384 kernels of size $3 \times 3 \times 256$ connected to the (normalized, pooled) outputs of the second convolutional layer. The fourth convolutional layer has 384 kernels of size $3 \times 3 \times 192$, and the fifth convolutional layer has 256 kernels of size $3 \times 3 \times 192$. The fully-connected layers have 4096 neurons each.

<https://proceedings.neurips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf>

Next Lesson

- Worked CNN example: Weather clasification
- Transfer learning
- (AI safety)

